



Proven advantage of multiobjective evolutionary algorithms for problems with different degrees of conflict

Wei jie Zheng ^{a,b}

^a School of Computer Science and Technology, State Key Laboratory of Smart Farm Technologies and Systems, International Research Institute for Artificial Intelligence, Harbin Institute of Technology, Shenzhen, China

^b Pengcheng Laboratory, Shenzhen, China

ARTICLE INFO

Keywords:

Degree of conflict
Runtime analysis
Multiobjective evolutionary algorithms
Scalarization
 ϵ -constraint approach
Multiobjective optimization

ABSTRACT

The field of multiobjective evolutionary algorithms (MOEAs) often emphasizes its popularity for optimization problems with conflicting objectives. However, it is still theoretically unknown how MOEAs perform compared with typical approaches outside this field.

This paper conducts such a systematic theoretical comparison on problem classes with different degrees of conflict. With ONEMAXMIN_k depicting $k \in [0..n]$ degrees of conflict, we show the difficulties of two typical non-MOEA approaches, the scalarization (weighted-sum) and the ϵ -constraint approach. We prove that for any set of weights, the set of optima formed by the scalarization approach cannot cover its full Pareto front for $k > 2$. Although constrained problems constructed from ϵ -constraint approach ensure the full coverage, general ways (via exterior or nonparameter penalty functions) to solve these constrained problems encounter difficulties. The nonparameter penalty function way cannot guarantee the full coverage, and the exterior way covers the Pareto front with expected $O(\max\{k, 1\}n \ln n)$ number of function evaluations, but only with careful settings of ϵ and r ($r > 1/(\epsilon + 1 - \lceil \epsilon \rceil)$).

In contrast, MOEAs efficiently solve ONEMAXMIN_k without careful designs. We prove the same expected runtime of $O(\max\{k, 1\}n \ln n)$ for the (G)SEMO, MOEA/D, NSGA-II, and SMS-EMOA.

Our brief discussions on a bi-objective LEADINGONES variant with different degrees of conflict show similar findings.

1. Introduction

Many real-world applications have multiple objectives to optimize simultaneously, and multiobjective evolutionary algorithms (MOEAs) are widely utilized to solve them. The theory community mainly uses the mathematical runtime as a tool to evaluate the performance of MOEAs rigorously, and to understand the working principles better. The runtime analysis of MOEAs starts from seminal papers by Laumanns et al. [1,2] and has made the breakthrough from toy algorithms (GSEMO) to practical algorithms like the NSGA-II [3,4] and its non-dominated sorting variants (NSGA-III [5,6], SMS-EMOA [7]) as well as the MOEA/D [8]. However, there are two fundamental questions that are still not well answered from the theoretical side.

As is known, MOEAs are not the only way to solve multiobjective optimization problems. Typical alternatives include scalarization (weighted-sum) and the ϵ -constraint method. The scalarization method combines all objectives into an integrated one via assigning weights to each objective, and then solves this single-objective optimization problem. The ϵ -constraint method selects one objective to optimize and converts remaining objectives into inequality constraints (with constants $\epsilon_1, \epsilon_2, \dots$ as corresponding lower bounds

E-mail address: zhengweijie@hit.edu.cn

<https://doi.org/10.1016/j.artint.2026.104573>

Received 31 March 2025; Received in revised form 11 May 2026; Accepted 29 May 2026

Available online 1 June 2026

0004-3702/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

for maximization problems), and then solves this constrained problem. However, the existing results for multiobjective optimization in our theory community mainly focus on the runtime of the MOEAs. The systematic theoretical comparison between MOEAs and typical non-MOEA ways is still not clear.

The field of MOEAs often emphasizes its popularity for optimization problems with conflicting objectives. The objectives can have different degrees of conflict when the decision variables take different values. For example, delivery personnel are required to transport spill-prone liquid foods, like coffee, soups, and congee, to customers rapidly and without leakage or quality deterioration. On high-quality road segments, delivery-time minimization and quality deterioration minimization are mutually non-conflicting objectives. On poor road segments, the two objectives conflict: detours maintain absence of spillage but lengthen travel time, whereas direct routes shorten time at the cost of increased spillage. There is a natural question about the relationship between the performance of the MOEAs and the conflict degree of the objectives, compared with other non-MOEA ways discussed before. Even a basic question exists about how MOEAs behave for problems with no conflict, compared to single-objective optimization algorithms.

This paper takes such a step to tackle the above two questions. We resort to the well-analyzed ONEMINMAX and COCZ benchmarks where the former considers the extreme conflict between two objectives, and the two objectives of the latter conflict in half of the bit positions. We consider a generalized class called ONEMAXMIN_k, where k is the number of bits in which two objectives conflict, and we call it the degree of conflict. This benchmark class includes ONEMINMAX for $k = n$ (n the problem size), $k = n/2$ for COCZ, and $k = 0$ for no conflict. We note that a similar function was independently proposed in [9], and we will discuss its details in Sections 2.2 and 3.

For this class of problems, we will show that two typical non-MOEA approaches, scalarization (weighted-sum) and ϵ -constraint approach, are difficult or inconvenient to use. We prove that for any set of weights, the set of optima (one optimum corresponds to one constructed single-objective problem) found by the scalarization approach cannot cover the full Pareto front of ONEMAXMIN_k with $k > 2$ (Theorem 1). Although the optima set of constrained problems constructed via ϵ -constraint approach can cover the full Pareto front, the commonly used ways (via exterior or nonparameter penalty functions) to solve such constrained problems encounter difficulties. The penalty function approach transforms the constrained problem into an unconstrained one. The nonparameter penalty function approach adds the unscaled violation to the objective when a constraint is violated, leaving it unchanged otherwise, and the exterior penalty function performs the same augmentation, but first scales the violation by a positive penalty coefficient. We will prove that the nonparameter penalty function way cannot guarantee the full coverage, and the exterior way helps (with expected runtime of $O(n \ln n)$ for the randomized local search algorithm for reaching any Pareto front point, see Theorem 2) but with careful settings of ϵ and r ($r > 1/(\epsilon + 1 - \lceil \epsilon \rceil)$) (Corollaries 2 and 3). Specifically, since the Pareto front size is $k + 1$, $k + 1$ different settings of ϵ are needed for the exterior way to achieve full coverage of the Pareto front, which results in a $O(\max\{k, 1\} n \ln n)$ runtime for the randomized local search algorithm.

In contrast to the careful designs required in the above-mentioned non-MOEA approaches, we will prove that in the typical settings, the popular toy MOEAs (the (G)SEMO), and the practical ones (the MOEA/D, the NSGA-II, and the SMS-EMOA) cover the full Pareto front of ONEMAXMIN_k in expected $O(\max\{k, 1\} n \ln n)$ function evaluations (Theorems 3 and 4), same as the exterior way. In particular, for solving the problem with two identical objectives ($k = 0$), the MOEAs will achieve the same runtime as the counterpart RLS or $(1 + 1)$ EA for single-objective optimization.

These results for the MOEAs hold for the following natural parameter settings. The result for the (G)SEMO does not have specific settings. The ones for the practical ones hold when the population size used in the NSGA-II and the SMS-EMOA is $\Theta(k + 1)$, the reference point used in SMS-EMOA is dominated by all Pareto optimal solutions, and the number of subproblems in MOEA/D is set to be the size of the Pareto front. Increasing population size is a generally used strategy (although the recent theoretical result [10] shows its inefficiency for more than two objectives on ONEMINMAX variants), and a good runtime ($O(N n \log n)$ for the NSGA-II, and $O(\mu n \log n)$ for the SMS-EMOA where N and μ are population sizes respectively) is still reached for a reasonably large population size. The restriction on the reference point is not that difficult to fulfill if one chooses a vector with small enough components for the maximization problem. Although the requirement on the number of subproblems in the MOEA/D seems strict, the very recent work [11] shows that starting from the coverage of a subset of the Pareto front, the MOEA/D still covers the full Pareto front in a reasonable runtime for the ONEMINMAX problem. Now we compare them with parameters used in the efficient exterior way to solve the constrained problems constructed from the ϵ -constraint approach, which also achieves an efficient performance. With respect to k degrees of conflict, this method needs to construct a specific set of ϵ parameters (with size at least $k + 1$) in a good distribution, while the NSGA-II and the SMS-EMOA only require their population size at least $4(k + 1)$ and $k + 1$ respectively, and the MOEA/D requires the number of subproblems being $k + 1$. Besides, this method also needs another positive penalty parameter r larger than $1/(\epsilon + 1 - \lceil \epsilon \rceil)$.

To see the generality of the above findings, we will have brief discussions on a bi-objective LEADINGONES variant with different degrees of conflict, called LOTZ_k with k for the conflict degree. The difficulty of the scalarization and the inconvenience of the ϵ -constraint method are observed as well. The MOEAs ((G)SEMO, NSGA-II, and SMS-EMOA) will be proved to have the same runtime of $O(\max\{k, 1\} n^2)$ as the ϵ -constraint method solved by the exterior penalty way with proper coefficients.

The remainder of this paper is organized as follows. Section 2 introduces the preliminaries of bi-objective pseudo-Boolean optimization and a brief review of the bi-objective benchmarks that are commonly used in the theory community. The ONEMAXMIN_k benchmark class is in Section 3 to depict the different degrees of conflict between two objectives. Sections 4 and 5 show the difficulties or inconvenience of two typical non-MOEA approaches, and the theoretical efficiency of commonly analyzed MOEAs is discussed in Section 6. The performance comparison on a bi-objective LEADINGONES variant is briefly discussed in Section 7. Section 8 concludes this paper.

2. Preliminaries

Following the standard notation, for $a, b \in \mathbb{Z}$ and $a \leq b$, we write $[a..b]$ to denote $[a, b] \cap \mathbb{Z}$. For $x = (x_1, \dots, x_n) \in \{0, 1\}^n$, we use $x_{[a..b]}$ to denote $(x_a, \dots, x_b)$, use $|x|_1$ to denote the number of ones in x , and abbreviate a sub-bitstring $(1, \dots, 1)$ with the length of k by 1^k . Besides, for any two vectors $u = (u_1, \dots, u_\ell), v = (v_1, \dots, v_\ell) \in \mathbb{R}^\ell$ with $\ell \in \mathbb{N}$, we say $u > (\geq, <, \leq, =) v$ if $u_i > (\geq, <, \leq, =) v_i$ for all $i \in [1..\ell]$.

2.1. Bi-objective pseudo-Boolean optimization

This work only considers bi-objective pseudo-Boolean optimization (maximization). That is, we consider to maximize $f = (f_1, f_2) : \{0, 1\}^n \rightarrow \mathbb{R}^2$. Different from single-objective optimization, not all solutions in multiobjective optimization are comparable. We say x *weakly dominates* y , denoted by $x \geq y$, if and only if $f_i(x) \geq f_i(y)$ for all $i \in \{1, 2\}$. We say x *dominates* y , denoted by $x > y$, if and only if $f_i(x) \geq f_i(y)$ for all $i \in \{1, 2\}$ and at least one of the inequalities is strict. We say x and y *incomparable* if and only if neither $x \geq y$ nor $y \geq x$. If x cannot be dominated by any other solutions, then we say x is *Pareto optimal*, and all Pareto optimal solutions form the *Pareto set*. The set of the function values of the Pareto set is called the *Pareto front*, and we call a vector on the Pareto front a *Pareto front point*. The *runtime* commonly discussed in the evolutionary theory community is the number of iterations or function evaluations to reach a predefined goal, like the full Pareto front coverage, that is, to reach a set of solutions whose function values contain all Pareto front points.

2.2. Literature review on bi-objective benchmarks used for theoretical analysis

The popular bi-objective benchmarks are the counterparts of the well-analyzed single-objective problems. Based on the classic single-objective ONEMAX benchmark that counts the number of ones in the bit string, the bi-objective COCZ [1] has ONEMAX as the first objective, and the second objective aligns with the first in the first half of the bit string but conflicts with it in the second half. Another counterpart ONEMINMAX [12] has ONEMAX as one objective and the other objective that conflicts in all bits (that is, this objective is ONEMAX w.r.t. $\bar{x} = 1 - x$). For the classic single-objective LEADINGONES benchmark, the bi-objective LOTZ [2] contains LEADINGONES as one objective and the other objective of a variant of LEADINGONES w.r.t. $\bar{x} = 1 - x$ but calculates from right to left. WLPTNO [13] generalizes LOTZ by shifting the search space from $\{0, 1\}^n$ to $\{-1, 1\}^n$ and simultaneously maximizing the leading positive ones and the trailing negative ones with weights. The popular multimodal JUMP class was used to be one objective in OJZJ [14], and OJZJ has the other objective of JUMP w.r.t. $\bar{x} = 1 - x$. DLTB [15] is the bi-objective counterpart of multimodal DLB (a deceptive version of LEADINGONES where the critical block (two positions) prefers the deceptive value of 00 instead of 11 before the optimal 1^n is reached) and is constructed in a similar way as LOTZ. These bi-objective benchmarks are well and intensively studied in the theory community [3,4,16–25]. Some artificial benchmarks are also constructed for specific aims. RR and URR [26] inspired by the single-objective royal road functions are designed to demonstrate the possible exponential speed-up of employing the crossover operator. ONETRAPHZERTRAP [27] is the bi-objective counterpart of the TRAP benchmark, with the first objective of the original trap and the other objective of TRAP w.r.t. $\bar{x} = 1 - x$.

From the construction of these benchmarks, we know that the conflict of two objectives exists and usually in an extreme manner. Very recently, independent of our arXiv version of this work [28], Antipov, Kötzing, and Radhakrishnan [9] generalized the COCZ and ONEMINMAX to the minimization of (OM_a, OM_b) where a and b are two optimal bitstrings w.r.t. two objectives respectively, and $OM_s(x)$ calculates the Hamming distance between x and s for $s \in \{a, b\}$. Here the Hamming distance between a and b can be regarded as the degree of conflicts. They also proved the expected runtime of $O(kn \ln n)$ for (G)SEMO, which partially answered our second question on the relationship between the performance of the (G)SEMO and the conflict degrees of objectives. However, our main question about the systematic comparison between typical non-MOEAs and MOEAs (including practical ones) is still not answered. Besides, they also considered a linear function class denoted by LFC where an offset is added and each bit is accompanied with a weight, and called the bit conflicted for two LFC with different weights if two corresponding weights are non-zero and have different signs. For the bi-objective problems constructed with LFC, no relevant theoretical results are given.

3. ONEMAXMIN_k

In this section, we will introduce the bi-objective benchmark class depicting the different degrees of the conflict between two objectives.

As discussed in Section 2.2, independent of ours, Antipov, Kötzing, and Radhakrishnan [9] considered different degrees of conflict for two objectives, generalized from COCZ and ONEMINMAX. Formally, they consider the minimization of

$$(OM_a, OM_b), \quad (1)$$

where $a, b \in \{0, 1\}^n$, and the function $OM_d : \{0, 1\}^n \rightarrow \mathbb{R}$ is defined by $OM_d(x) = H(d, x)$, that is, the Hamming distance between two bitstrings d and x . For this problem, $k = H(a, b)$ can be regarded as the degree of conflict. We omit the introduction about a more general conflict defined on the generalized linear function class LFC, as no theoretical results are given for this kind and our results will focus on more intuitive and easy problems but conjecture our findings will be helpful for the future analysis.

To better fit into the structures of COCZ and ONEMINMAX, we now introduce the essentially identical but more intuitive benchmark class, comparing to (1). We note that the obtained results in this work can be easily applied to the minimization of (1). Following

the similar structure in COCZ, the following benchmark class makes the first and the second objectives concur in the first $n - k$ bit positions and conflict in the last k bit positions. k is the degree of conflict between two objectives. See Definition 1.

Definition 1. The ONEMAXMIN_k function $\{0, 1\}^n \rightarrow \mathbb{R}^2$ is defined by

$$f^k(x) = (f_1^k(x), f_2^k(x)) = \left(\sum_{j=1}^n x_j, \left(\sum_{j=1}^{n-k} x_j \right) + \left(\sum_{j=n-k+1}^n 1 - x_j \right) \right) \quad (2)$$

for $x = (x_1, \dots, x_n) \in \{0, 1\}^n$.

We easily see that it includes COCZ for $k = n/2$, and ONEMINMAX (swapping the orders of two objectives) for $k = n$. The following lemma shows the Pareto front for ONEMAXMIN_k.

Lemma 1. Let M denote the Pareto front of ONEMAXMIN_k. Then

$$M = \{(n - k + i, n - i) \mid i \in [0..k]\}.$$

Proof. We first show that $y \not\succ x$ for any x with $f^k(x) \in M$ and for any $y \in \{0, 1\}^n$. If $y \succ x$, from the definition of dominance, we know

$$f_1^k(y) + f_2^k(y) > f_1^k(x) + f_2^k(x). \quad (3)$$

From (2) we have

$$f_1^k(y) + f_2^k(y) = 2|y_{[1..n-k]}|_1 + k \leq 2(n - k) + k = 2n - k.$$

Noting that $f_1^k(x) + f_2^k(x) = 2n - k$ from the definition of M , we know

$$f_1^k(y) + f_2^k(y) \leq f_1^k(x) + f_2^k(x),$$

which contradicts (3). Then $y \not\succ x$.

Now it remains to prove that for any $y \in \{0, 1\}^n$ with $f^k(y) \notin M$, there exists $z \in \{0, 1\}^n$ such that $z \succ y$. From $f^k(y) \notin M$, we easily see $|y_{[1..n-k]}|_1 \neq 1^{n-k}$. Let z with $z_{[1..n-k]} = 1^{n-k}$ and $z_{[n-k+1..n]} = y_{[n-k+1..n]}$. Then $f_1^k(z) > f_1^k(y)$ and $f_2^k(z) > f_2^k(y)$. Thus $z \succ y$. $\square$

In this work, we will discuss the runtime (number of function evaluations or iterations) that the algorithms need to cover the full Pareto front.

The following lemma calculates the maximal number of incomparable solutions, which will be used in the later sections.

Lemma 2. The maximal number of pairwise non-dominated function values w.r.t. ONEMAXMIN_k is $k + 1$.

Proof. Let V be the set of mutually incomparable solutions. We will show that for any incomparable $x, y \in V$, $|x_{[n-k+1..n]}|_1 \neq |y_{[n-k+1..n]}|_1$. If $|x_{[n-k+1..n]}|_1 = |y_{[n-k+1..n]}|_1$, noting that

$$\begin{cases} f^k(x) = (|x_{[1..n-k]}|_1 + |x_{[n-k+1..n]}|_1, |x_{[1..n-k]}|_1 + k - |x_{[n-k+1..n]}|_1) \\ f^k(y) = (|y_{[1..n-k]}|_1 + |y_{[n-k+1..n]}|_1, |y_{[1..n-k]}|_1 + k - |y_{[n-k+1..n]}|_1), \end{cases}$$

we have that

- if $|x_{[1..n-k]}|_1 \geq |y_{[1..n-k]}|_1$, then $f^k(x) \geq f^k(y)$, thus $x \geq y$;
- if $|x_{[1..n-k]}|_1 < |y_{[1..n-k]}|_1$, then $f^k(x) < f^k(y)$, thus $y \succ x$,

which contradicts that x and y are incomparable. Since $|x_{[n-k+1..n]}|_1 \in [0..k]$, we have $|V| \leq k + 1$. With the Pareto front size of $k + 1$ from Lemma 1, this lemma is proved. $\square$

4. Difficulty of the scalarization approach

In this section and the next section, we will discuss how the typical non-MOEA approaches (scalarization and ε -constraints) behave to different degrees of the objectives, that is, how they solve the ONEMAXMIN_k defined in Section 3. We will see the difficulties or the inconveniences of these two approaches.

4.1. Scalarization (weighted-sum)

One well-known approach to tackle multiobjective optimization is to use scalarization (or called weighted-sum approach) to combine all objectives into an integrated one via assigning weights to each [29]. Formally, the scalarization will reformulate the maximization of the original bi-objective ONEMAXMIN_k (Definition 1) to the maximization of the following function.

Definition 2. Let $w \in \mathbb{R}$. The scalarization function $f_w^k : \{0, 1\}^n \rightarrow \mathbb{R}$ of ONEMAXMIN_k is defined by

$$\begin{aligned} f_w^k(x) &= w f_1^k(x) + (1 - w) f_2^k(x) \\ &= \left(\sum_{j=1}^{n-k} x_j \right) + k(1 - w) + (2w - 1) \left(\sum_{j=n-k+1}^n x_j \right) \end{aligned} \quad (4)$$

for $x = (x_1, \dots, x_n) \in \{0, 1\}^n$.

Obviously, the optimum is 1^n for $2w - 1 > 0$, and $1^{n-k}0^k$ for $2w - 1 < 0$. For $2w - 1 = 0$, the optima are $1^{n-k} *$ with any $*$ $\in \{0, 1\}^k$.

4.2. Solving the constructed problems

Usually, an algorithm for one single-objective problem only provides one solution. Here we assume that the algorithm will be terminated when one Pareto optimum is reached. Then one problem contributes to at most one Pareto front point (it depends on whether this Pareto front point has already been reached or not). Hence, to cover the full Pareto front, we need to carefully select a set of weights w to construct the minimal number of problems (and also need to carefully select corresponding algorithms).¹ However, the suitable choices of weights are not known beforehand. We point out a worse situation in the following theorem that even using an infinite number of weights and different algorithms, the weighted-sum approach cannot fully cover the Pareto front of ONEMAXMIN_k . We note that from the proof of the following theorem, we see a positive chance for a coverage of the Pareto front when restarting $w = 1/2$ for many times. However, since one cannot easily have the prior knowledge of repeating a specific weight many times (and how many times should we repeat is not well answered as well), we will not discuss the possibility of this impractical case in this work.

Theorem 1. Let M be defined as in Lemma 1 and $k > 2$. Let $S \subseteq \mathbb{R}$ be a set of w , and x_w be one global optimum of f_w^k . Let $F = \{f^k(x_w) \mid w \in S\}$. Then $M \not\subseteq F$.

Proof. If there is $w \in S$ with $2w - 1 > 0$, then as discussed before, $x_w = 1^n$ and thus $(n, n - k) \in F$. Similarly, if there is $w \in S$ with $2w - 1 < 0$, then $x_w = 1^{n-k}0^k$ and thus $(n - k, n) \in F$. If $w = 1/2 \in S$, then $x_{1/2} \in \{1^{n-k} * | * \in \{0, 1\}^k\}$. Since only one optimal solution is picked for $f_{1/2}^k$, we know that $x_{1/2}$ will contribute at most one more Pareto front point into F , which depends on whether $x_{1/2} \in \{1^n, 1^{n-k}0^k\}$ or not. Hence, for any S , maximizing $f_{w \in S}^k$ reaches at most three Pareto front points, which is less than $|M| = k + 1$. $\square$

We note that the drawback of the weighted-sum approach has been already recognized for a long time. For example, in 1997, Das and Dennis [30] gave the geometrical explanation of two major drawbacks of this approach. The first drawback is that no weight will lead to solutions in the non-convex part of the non-convex Pareto curve. The second drawback is that the even spread of weights does not result in an even spread on the Pareto curve. In textbooks on multiobjective optimizations [31,32], the major drawback is regarded as “this approach cannot generate concave portions of the Pareto front regardless of the weights used” [32]. [31, Section 3.1] also listed other practical disadvantages, like the second drawback mentioned in [30]. Besides, with no example given, it was pointed out that multiple solutions w.r.t. a specific weight may result in a proper subset of Pareto set. Since a set with some but not all Pareto optima does not always mean an incomplete coverage of the Pareto front, it is a straightforward question whether all points in the convex part of the Pareto front curve can be covered with a set of mutually different weights (which can be uncountable). In fact, our Theorem 1 indicates a negative answer.

Note that the discrete solution space of $\{0, 1\}^n$ results in a non-convex domain of definition. In order to construct a convex Pareto front curve, we just need to change the domain of definition to $[0, 1]^n$. In this case, the Pareto front curve is a line segment of $\{(n - k + i, n - i) \mid i \in [0, k]\}$, which is obviously a convex set (from the definition of the convex set). With this domain of definition, the optimum for maximizing (4) is also 1^n for $2w - 1 > 0$, and $1^{n-k}0^k$ for $2w - 1 < 0$. For $2w - 1 = 0$, the optima are $1^{n-k} *$ with any $* \in [0, 1]^k$. Therefore, with the same proof, we note that Theorem 1 also holds for this domain of definition. That is, at most three Pareto front points will be reached for any set of weights. Noting the uncountable Pareto front points $\{(n - k + i, n - i) \mid i \in [0, k]\}$, we proved our claim that we cannot cover the convex part of the Pareto front cover by setting a set of mutually different weights (conditional on that one weight provides one solution).

5. Inconveniences of ε -constraint approach

The previous section shows the difficulty of using the scalarization approach to optimize the bi-objective ONEMAXMIN_k . In this section, we will discuss the inconveniences of another well-known way of the ε -constraint approach.

5.1. ε -constraint approach

The ε -constraint approach is also a well-known way to tackle the multiobjective optimization [29,31]. For the bi-objective maximization problem, the ε -constraint approach selects one objective to optimize and requires the other objective not smaller than ε as a constraint. Formally, this approach reformulates (2) into the following problem (w.l.o.g., treating f_2^k as a constraint).

$$\begin{aligned} \max f_1^k(x) &= \sum_{j=1}^n x_j \\ \text{s.t. } f_2^k(x) &= \left(\sum_{j=1}^{n-k} x_j \right) + \left(\sum_{j=n-k+1}^n 1 - x_j \right) \geq \varepsilon \end{aligned} \quad (5)$$

¹ Note here that a sequence of solutions, say $x_{(w)}^1, \dots, x_{(w)}^{T_w}$, will be generated in the optimization process of each single-objective problem w.r.t. w . For a set of weights $\{w_1, \dots, w_K\}$ for some $K \in \mathbb{N}$, this work only consider whether the set of final returned solutions $\{x_{(w_1)}^{T_{w_1}}, \dots, x_{(w_K)}^{T_{w_K}}\}$, instead of the historically generated ones $\{x_{(w_1)}^1, \dots, x_{(w_1)}^{T_{w_1}}, \dots, x_{(w_K)}^1, \dots, x_{(w_K)}^{T_{w_K}}\}$, covers the full Pareto front.

For each ε , solving (5) by an algorithm provides one solution. Recall that our goal for (2) is to obtain a diverse set of solutions whose function values cover the whole Pareto front of f^k . Hence, with the ε -constraint approach, we need to choose a minimal number (at least $k + 1$) of ε instances to meet our goal.

We now discuss the optimal solutions for different instances of ε . Since $\max f_2^k = n$, we know that no feasible solution exists for $\varepsilon > n$. If $\varepsilon \leq n - k$, then the optimal solution is 1^n . If $\varepsilon \in (n - k, n]$, let $\eta = f_2^k(x) = |x_{[1..n-k]}|_1 + k - |x_{[n-k+1..n]}|_1$, then

$$\begin{aligned} f_1^k(x) &= |x_{[1..n-k]}|_1 + |x_{[n-k+1..n]}|_1 \\ &= |x_{[1..n-k]}|_1 + (|x_{[1..n-k]}|_1 + k - \eta) \\ &= 2|x_{[1..n-k]}|_1 + k - \eta. \end{aligned}$$

Hence, the maximal f_1^k value of $2n - k - \lceil \varepsilon \rceil$ among all feasible solutions is reached when $|x_{[1..n-k]}|_1 = n - k$ and $\eta = \lceil \varepsilon \rceil$, that is, when $x_{[1..n-k]} = 1^{n-k}$ and $|x_{[n-k+1..n]}|_1 = n - \lceil \varepsilon \rceil$. We note that not all solutions with f_1^k value of $2n - k - \lceil \varepsilon \rceil$ are feasible. Together with the Pareto front of ONEMAXMIN_k in Lemma (1), we know that for a set of constrained problems (5) with $\varepsilon = \varepsilon_i, i \in [0..k]$ where $\varepsilon_i \in (n - k + i - 1, n - k + i], i \in [1..k]$ and $\varepsilon_0 \in (-\infty, n - k]$, the set of optimal solutions results in a full coverage for ONEMAXMIN_k.

5.2. Solving the constrained problem

Now we consider solving the constrained optimization problem (5). For evolutionary algorithms, the most used way is based on the penalty functions [33]. With penalty functions, the constrained problem is reformed as an unconstrained problem to solve. Following [33], we discuss the following two kinds of penalty functions: an exterior penalty function and a nonparameter penalty function.

5.2.1. Exterior penalty function

To penalize the constraint when it is violated, a simple exterior penalty function with penalty coefficient $r > 0$ for (5) can be constructed as maximizing

$$g(x) = f_1^k(x) + r \min\{0, f_2^k(x) - \varepsilon\}. \quad (6)$$

If the solution x is infeasible (that is, $f_2^k(x) < \varepsilon$), then (6) becomes $f_1^k(x) - r(\varepsilon - f_2^k(x))$, which penalizes $f_1^k(x)$ by the absolute difference with a factor of r . If x is feasible, then $g(x) = f_1^k(x)$.

A. Optimal Solution Set

It is not difficult to see that the optimal solution set for maximizing (6) depends on different values of r and ε . To obtain the optimal solution set for $\varepsilon \in (n - k, n)$ in Lemma 4, we need a complete (and complicated) calculation of the maximum among $A, B, \max_{i \in I} K_i$ defined as in the following lemma. Hence, for a clear demonstration in the proof of Lemma 4, we extract this complicated calculation here.

Lemma 3. Let $n, k \in \mathbb{N}, n \geq k, r > 0$ and $\varepsilon \in (n - k, n)$. Let $r_1 = \frac{\lceil \varepsilon \rceil - (n - k)}{\varepsilon - (n - k)}, r_2 = \frac{1}{\varepsilon + 1 - \lceil \varepsilon \rceil}, A := 2n - k - \lceil \varepsilon \rceil, B := n + r(n - k - \varepsilon), K_i := 2n - k - r\varepsilon + (r - 1)i$ for $i \in [n - k + 1.. \lceil \varepsilon \rceil - 1] =: I$ (note that $I = \emptyset$ if $n - k + 1 > \lceil \varepsilon \rceil - 1$, that is, if $\varepsilon \in (n - k, n - k + 1]$), and $C := K_{\lceil \varepsilon \rceil - 1}$. Then $1 \leq r_1 \leq r_2$ holds, where $r_1 = r_2$ iff $\varepsilon \in [n - k + 1..n - 1] \cup (n - k, n - k + 1)$, and $r_1 = 1$ iff $\varepsilon \in [n - k + 1..n - 1]$. Besides, we have

$$\begin{aligned} &\max \left\{ A, B, \max_{i \in I} K_i \right\} \\ &= \begin{cases} B, & \text{if } (\varepsilon \in (n - k, n), r \in (0, 1)), \\ K_{i \in I} = B, & \text{if } (\varepsilon \in (n - k, n) \setminus \mathbb{N}, r = 1), \\ K_{i \in I} = B = A, & \text{if } (\varepsilon \in (n - k, n) \cap \mathbb{N}, r = 1), \\ C = B, & \text{if } (\varepsilon \in (n - k, n - k + 1], r \in (1, r_1)), \\ C, & \text{if } (\varepsilon \in (n - k + 1, n), r \in (1, r_1)), \\ C = B = A, & \text{if } (\varepsilon \in (n - k, n - k + 1), r = r_1 > 1), \\ C, & \text{if } (\varepsilon \in (n - k + 1, n) \setminus \mathbb{N}, r = r_1 > 1), \\ C, & \text{if } (\varepsilon \in (n - k, n), r \in (r_1, r_2)), \\ C = A, & \text{if } (\varepsilon \in (n - k + 1, n) \setminus \mathbb{N}, r = r_2 > r_1), \\ A, & \text{if } (\varepsilon \in (n - k, n), r > r_2). \end{cases} \end{aligned}$$

Note here that the above calculation is complete. For example, " $K_{i \in I} = B$, if $(\varepsilon \in (n - k, n) \setminus \mathbb{N}, r = 1)$ " means that when $\varepsilon \in (n - k, n) \setminus \mathbb{N}$ and $r = 1$,

$$\max \left\{ A, B, \max_{i \in I} K_i \right\} = K_{i \in I} = B > A$$

for any $i \in I$.

Proof. It is not difficult to see that $r_1 \geq 1$ and the equality holds iff $\varepsilon \in \mathbb{N}$. Since $\lceil \varepsilon \rceil \geq n - k + 1$, we know that $\varepsilon - (n - k) \geq \varepsilon + 1 - \lceil \varepsilon \rceil > 0$, and thus

$$\begin{aligned} r_2 - r_1 &= \frac{1}{\varepsilon + 1 - \lceil \varepsilon \rceil} - \frac{\lceil \varepsilon \rceil - (n - k)}{\varepsilon - (n - k)} \\ &= \frac{\lceil \varepsilon \rceil - \varepsilon}{\varepsilon + 1 - \lceil \varepsilon \rceil} - \frac{\lceil \varepsilon \rceil - \varepsilon}{\varepsilon - (n - k)} \geq 0, \end{aligned}$$

where the equality holds iff $\varepsilon = \lceil \varepsilon \rceil$ or $\lceil \varepsilon \rceil = n - k + 1$, that is, $\varepsilon \in \mathbb{N} \cup (n - k, n - k + 1]$. Thus $r_2 \geq r_1 \geq 1$, and we also know that $r_2 = r_1 = 1$ holds iff $\varepsilon \in \mathbb{N}$, and $r_2 = r_1 > 1$ holds iff $\varepsilon \in (n - k, n - k + 1)$.

We first note that

$$\begin{aligned} K_{\lceil \varepsilon \rceil - 1} &= 2n - k - r\varepsilon + (r - 1)(\lceil \varepsilon \rceil - 1) \\ &= 2n - k - (\lceil \varepsilon \rceil - 1) + r(\lceil \varepsilon \rceil - 1 - \varepsilon) = C, \end{aligned} \quad (7)$$

and that

$$\begin{aligned} B - C &= n + r(n - k - \varepsilon) - (2n - k - (\lceil \varepsilon \rceil - 1) + r(\lceil \varepsilon \rceil - 1 - \varepsilon)) \\ &= k + \lceil \varepsilon \rceil - 1 - n + r(n - k - \varepsilon - (\lceil \varepsilon \rceil - 1 - \varepsilon)) \\ &= k + \lceil \varepsilon \rceil - 1 - n + r(n - k - \lceil \varepsilon \rceil + 1) \\ &= (r - 1)(n - k - \lceil \varepsilon \rceil + 1). \end{aligned} \quad (8)$$

For $r < r_1 = \frac{\lceil \varepsilon \rceil - (n - k)}{\varepsilon - (n - k)}$, we have

$$\begin{aligned} B &= n + r(n - k - \varepsilon) > n + \frac{\lceil \varepsilon \rceil - (n - k)}{\varepsilon - (n - k)}(n - k - \varepsilon) \\ &= n + n - k - \lceil \varepsilon \rceil = 2n - k - \lceil \varepsilon \rceil = A, \end{aligned} \quad (9)$$

where the first inequality uses $\varepsilon > n - k$. In this case, we further consider $r \in (0, 1)$, $r = 1$, and $r \in (1, r_1)$ respectively. If $r \in (0, 1)$, then

$$\begin{aligned} \max_{i \in I} K_i &= \max_{i \in I} (2n - k - r\varepsilon + (r - 1)i) \\ &= 2n - k - r\varepsilon + (r - 1)(n - k + 1) = n - 1 + r(n - k + 1 - \varepsilon) \\ &= n + r(n - k - \varepsilon) + r - 1 < n + r(n - k - \varepsilon) = B. \end{aligned}$$

With (9), the result for $r \in (0, 1)$ is shown.

If $r = 1$, then

$$\begin{aligned} \max_{i \in I} K_i &= \max_{i \in I} (2n - k - r\varepsilon + (r - 1)i) \\ &= K_{i \in I} = 2n - k - \varepsilon (= n + r(n - k - \varepsilon) = B) \\ &\geq 2n - k - \lceil \varepsilon \rceil = A, \end{aligned}$$

where the equality in the last inequality holds iff $\varepsilon \in \mathbb{N}$. Hence, we have the results for $r = 1$.

If $r \in (1, r_1)$, which already requires $\varepsilon \notin \mathbb{N}$ to ensure $r_1 > 1$ as we proved before, then

$$\max_{i \in I} K_i = \max_{i \in I} (2n - k - r\varepsilon + (r - 1)i) = C \geq B, \quad (10)$$

where the last equality uses (7), and the last inequality uses (8) ≤ 0 since $r > 1$ and $\lceil \varepsilon \rceil \geq n - k + 1$, and thus the equality in this inequality holds iff $\lceil \varepsilon \rceil = n - k + 1$, that is, $\varepsilon \in (n - k, n - k + 1]$. Hence, we have the results for $r \in (1, r_1)$. Note also that (10) holds for all $r > 1$.

For $r = r_1 = \frac{\lceil \varepsilon \rceil - (n - k)}{\varepsilon - (n - k)}$, we have

$$B = n + r(n - k - \varepsilon) = 2n - k - \lceil \varepsilon \rceil = A.$$

With (10) and $r_1 > 1$, we have the results for $r = r_1$.

For $r > r_1 = \frac{\lceil \varepsilon \rceil - (n - k)}{\varepsilon - (n - k)}$, similar to (9), we have

$$B < A. \quad (11)$$

In this case, we further consider $r \in (r_1, r_2)$, $r = r_2$, and $r > r_2$ respectively. If $r \in (r_1, r_2)$, which already requires $\varepsilon \in (n - k + 1, n) \setminus \mathbb{N}$ to ensure $r_2 > r_1$ as we proved before, then with (7) we have

$$C > 2n - k - (\lceil \varepsilon \rceil - 1) + \frac{\lceil \varepsilon \rceil - 1 - \varepsilon}{\varepsilon + 1 - \lceil \varepsilon \rceil} = 2n - k - \lceil \varepsilon \rceil = A,$$

where the first inequality uses $\lceil \varepsilon \rceil - 1 - \varepsilon < 0$ and $r < r_2 = \frac{1}{\varepsilon + 1 - \lceil \varepsilon \rceil}$. Together with (11), the result for $r \in (r_1, r_2)$ is proved.

Similarly, if $r = r_2$, we have $C = A$. With (11) and $r_2 > r_1$, we have the result for $r = r_2$.

Also, if $r > r_2$, we have $C < A$. With (11), we have the result for $r > r_2$. $\square$

Now we have the following lemma for the optimal solution set for different penalty coefficients r , and different relationships between ε and n .

Lemma 4. Let A, B, C, I and $K_i, i \in I$ be defined as in Lemma 3. Let $\varepsilon' = \lceil \varepsilon \rceil - 1$ and $D_i = \{x \mid f^k(x) = (2n - k - i, i)\}$ (that is, $D_i = \{x \mid x_{[1..n-k]} = 1^{n-k}, |x_{[n-k+1..n]}|_1 = n - i\}$) for $i = n - k, \dots, n$. Let the penalty coefficient $r > 0$, and let S be the optimal solution set of maximizing (6). Then

$$S = \begin{cases} D_n, & \text{if } (\varepsilon \geq n, r > 1), \\ \cup_{i=n-k}^n D_i, & \text{if } (\varepsilon \geq n, r = 1), \\ D_{n-k}, & \text{if } \varepsilon \leq n - k \text{ or } (\varepsilon \in (n - k, +\infty), r \in (0, 1)), \\ \cup_{i=n-k}^{\lceil \varepsilon \rceil - 1} D_i, & \text{if } (\varepsilon \in (n - k, n), \varepsilon \notin \mathbb{N}, r = 1), \\ \cup_{i=n-k}^{\lceil \varepsilon \rceil} D_i, & \text{if } (\varepsilon \in (n - k, n), \varepsilon \in \mathbb{N}, r = 1), \\ D_{n-k}, & \text{if } (\varepsilon \in (n - k, n - k + 1], r \in (1, r_1)), \\ D_{\varepsilon'}, & \text{if } (\varepsilon \in (n - k + 1, n), r \in (1, r_1)) \text{ or} \\ & (\varepsilon \in (n - k + 1, n) \setminus \mathbb{N}, r = r_1 > 1) \text{ or} \\ & (\varepsilon \in (n - k, n), r \in (r_1, r_2)), \\ D_{n-k} \cup D_{\lceil \varepsilon \rceil}, & \text{if } (\varepsilon \in (n - k, n - k + 1), r = r_1 > 1), \\ D_{\varepsilon'} \cup D_{\lceil \varepsilon \rceil}, & \text{if } (\varepsilon \in (n - k + 1, n) \setminus \mathbb{N}, r = r_2 > r_1), \\ D_{\lceil \varepsilon \rceil}, & \text{if } (\varepsilon \in (n - k, n), r > r_2). \end{cases}$$

Proof. If $\varepsilon \geq n$, since $\max_{x \in \{0,1\}^n} f_2^k(x) = f_2^k(1^{n-k}0^k) = n$, we know that for any x ,

$$\begin{aligned} g(x) &= f_1^k(x) + r(f_2^k(x) - \varepsilon) = f_1^k(x) + r f_2^k(x) - r\varepsilon \\ &= \sum_{j=1}^n x_j + r \left(\left(\sum_{j=1}^{n-k} x_j \right) + \left(\sum_{j=n-k+1}^n 1 - x_j \right) \right) - r\varepsilon \\ &= (1+r) \left(\sum_{j=1}^{n-k} x_j \right) + (1-r) \left(\sum_{j=n-k+1}^n x_j \right) + r(k - \varepsilon). \end{aligned} \quad (12)$$

Then it is not difficult to see the set of optimal solutions is $\{1^{n-k}0^k\} = D_n$ if $r > 1$, $\cup_{i=n-k}^n D_i$ if $r = 1$, and D_{n-k} if $r \in (0, 1)$. If $\varepsilon \in (n - k, n)$, then

$$g(1^n) = n + r \min\{0, n - k - \varepsilon\} = n + r(n - k - \varepsilon) = B.$$

Recall that $\max_{x \in \{0,1\}^n} f_2^k(x) = f_2^k(1^{n-k}0^k) = n$. Note that for $i \in [n - k..n]$,

$$\begin{aligned} \max_{x \in \{z \mid f_2^k(z) = i\}} f_1^k(x) &= n - k + (k - (i - (n - k))) \\ &= n + (n - k - i) = 2n - k - i \end{aligned} \quad (13)$$

decreases when i increases. Hence, we know that for any $x \in \{0, 1\}^n$ with $f_2^k(x) \geq \varepsilon$, we have

$$g(x) = f_1^k(x) \leq 2n - k - \lceil \varepsilon \rceil = A, \quad (14)$$

where under the condition $f_2^k(x) \geq \varepsilon$, $g(x) = A$ holds iff $x \in \{z \mid f^k(z) = (2n - k - \lceil \varepsilon \rceil, \lceil \varepsilon \rceil)\} = D_{\lceil \varepsilon \rceil}$. Also note that for $i \in [0..n - k]$,

$$\max_{x \in \{z \mid f_2^k(z) = i\}} f_1^k(x) = i + k$$

increases when i increases. Hence, together with (13) we know that for any $x \in \{0, 1\}^n$ with $f_2^k(x) < \varepsilon$, we have

$$\begin{aligned} g(x) &= f_1^k(x) + r(f_2^k(x) - \varepsilon) \\ &\leq \max \left\{ \max_{i \in [0..n-k]} (i + k + r(i - \varepsilon)), \max_{i \in I} (2n - k - i + r(i - \varepsilon)) \right\} \\ &= \max \left\{ n - k + k + r(n - k - \varepsilon), \max_{i \in I} K_i \right\} \\ &= \max \left\{ B, \max_{i \in I} K_i \right\}, \end{aligned} \quad (15)$$

where the first equality uses the definition of K_i and the fact that the maximum of $i + k + r(i - \varepsilon)$ for $i \in [0..n - k]$ is reached iff $i = n - k$, that is, $x = 1^n$. With (14) and (15), we know that the maximal function value of g is

$$\max \left\{ A, B, \max_{i \in I} K_i \right\}.$$

Note that in the above discussion, we require the only case $D_{\lceil \varepsilon \rceil}$ to let A be reachable, D_{n-k} for B , and D_i for K_i with $i \in [n-k+1, \lceil \varepsilon \rceil - 1]$ (including D_{ε} for C). Then from Lemma 3, the case for $\varepsilon \in (n-k, n)$ is proved.

If $\varepsilon \leq n-k$, then

$$g(1^n) = n + r \min\{0, n-k-\varepsilon\} = n.$$

For any $x \in \{0, 1\}^n \setminus \{1^n\}$, we know that $f_1^k(x) < n$, and thus

$$g(x) = f_1^k(x) + r \min\{0, f_2^k(x) - \varepsilon\} \leq f_1^k(x) < n = g(1^n).$$

That is, the optimal solution is 1^n . $\square$

Recall that for a certain $\varepsilon \in (n-k, n]$, the optimal solution set of (5) is $\{x \mid f^k(x) = (2n-k-\lceil \varepsilon \rceil, \lceil \varepsilon \rceil)\} = D_{\lceil \varepsilon \rceil}$, and the optimal set is $\{x \mid f^k(x) = (n, n-k)\} = D_{n-k}$ for $\varepsilon \leq n-k$ and is $\emptyset$ for $\varepsilon > n$. We have the following corollary for the comparison between the constrained problem (5) and the problem (6) constructed via the exterior penalty function.

Corollary 1. Let $r > 0$. Then the optimal solution sets of (5) and (6) are identical for $\varepsilon \in (-\infty, n-k]$, ($\varepsilon = n, r > 1$), and ($\varepsilon \in (n-k, n), r \in (\frac{1}{\varepsilon+1-\lceil \varepsilon \rceil}, +\infty)$), and are different for other cases.

B. Runtime Analysis

B1. Improper Parameter Settings for an Incomplete Coverage of the Pareto Front

With the above result, we can easily have the following corollary about the algorithm's difficulty in solving the exterior penalty problem with improper parameter settings, which results in an incomplete coverage the Pareto front of ONEMAXMIN $_k$.

Corollary 2. Let M be defined as in Lemma 1, $k \geq 1$, $r \in (0, 1)$, and $S \subseteq \mathbb{R}$ be a set of ε . Let x_ε be one optimal solution for the maximization of g with a certain $\varepsilon \in S$. Let $F = \{f^k(x_\varepsilon) \mid \varepsilon \in S\}$. Then $M \not\subseteq F$.

Proof. From Lemma 4, we know that for this case of $r \in (0, 1)$, $x_\varepsilon \in D_{n-k} = \{x \mid f^k(x) = (n, n-k)\}$. Hence, $F = \{(n, n-k)\}$ and thus $|F| = 1 < 2 \leq k+1 = |M|$. Then $M \not\subseteq F$. $\square$

B2. Runtime for Proper Parameter Settings

Luckily, from Lemmas 1 and 4, the following corollary shows that with careful settings, this approach can result in a full coverage of the Pareto front for (2).

Corollary 3. Let $\varepsilon_0 \in (-\infty, n-k]$, $\varepsilon_1 \in (n-k, n-k+1]$, $\dots$, $\varepsilon_{k-1} \in (n-2, n-1]$, and $\varepsilon_k \in (n-1, +\infty)$. Let M be defined as in Lemma 1.

- Consider to maximize (6) with $\varepsilon = \varepsilon_i$ and $r > \frac{1}{\varepsilon_i+1-\lceil \varepsilon_i \rceil}$ for any $i \in [0..k]$. Then D_{n-k+i} is the set of optimal solutions.
- Let G be any set with $\{\varepsilon_0, \dots, \varepsilon_k\} \subseteq G$ and $r > \max_{i=0, \dots, k} \frac{1}{\varepsilon_i+1-\lceil \varepsilon_i \rceil}$. Let x_ε be one optimal solution for the maximization of g with a certain $\varepsilon \in G$ and $F = \{f^k(x_\varepsilon) \mid \varepsilon \in G\}$. Then $M \subseteq F$.

With the above setting, now we analyze the runtime of the randomized local search algorithm to maximize (6) with $r > \frac{1}{\varepsilon+1-\lceil \varepsilon \rceil}$. The randomized local search (RLS) algorithm is a simple single-objective optimizer. It starts from a single randomly generated solution. In each generation, it applies the one-bit mutation (uniformly at random picking one bit and flipping its bit value) on the parent solution to generate an offspring solution. If this offspring has at least the same fitness as its parent, then the offspring will replace its parent and enter into the next generation. We first have the following two lemmas about the survival situations starting from an infeasible solution and from a feasible solution, respectively.

Lemma 5. Consider using the randomized local search algorithm to maximize g defined as in (6) with $r > \frac{1}{\varepsilon+1-\lceil \varepsilon \rceil}$. Then starting from an infeasible solution (whose f_2^k value is less than ε), only the offspring via flipping one 0 bit in the first $n-k$ positions or via flipping one 1 bit in the last k positions will survive to the next population.

Proof. Let x be an infeasible solution in the current population, y be the offspring generated by applying one-bit mutation to x , and j be the bit position where x and y are different. Let I_0, I_1 be the sets of all bit positions with the value of zero and the value of one in the first $n-k$ bits of x respectively, and let I'_0, I'_1 be the sets of all bit positions with the value of zero and the value of one in the last k bits of x respectively. If $j \in I_0$, then $f_1^k(y) > f_1^k(x)$, $f_2^k(y) > f_2^k(x)$, and thus $g(y) > g(x)$. Hence y will survive to the next generation.

If $j \in I_1$, then $f_1^k(y) < f_1^k(x)$, $f_2^k(y) < f_2^k(x)$, and thus $g(y) < g(x)$. Hence y will be removed.

If $j \in I'_0$, then $f_1^k(y) = f_1^k(x) + 1$ and $f_2^k(y) = f_2^k(x) - 1 < \varepsilon$ as x is infeasible. Then

$$g(y) - g(x) = f_1^k(y) + r(f_2^k(y) - \varepsilon) - (f_1^k(x) + r(f_2^k(x) - \varepsilon)) = 1 - r < 0,$$

where the last inequality uses $r > \frac{1}{\varepsilon+1-\lceil \varepsilon \rceil} > 1$. Hence, y will be removed.

If $j \in I'_1$, then $f_1^k(y) = f_1^k(x) - 1$ and $f_2^k(y) = f_2^k(x) + 1$. Then if y is infeasible, we have

$$g(y) - g(x) = f_1^k(y) + r(f_2^k(y) - \varepsilon) - (f_1^k(x) + r(f_2^k(x) - \varepsilon)) = r - 1 > 0,$$

where the last inequality uses $r > \frac{1}{\varepsilon+1-\lceil \varepsilon \rceil} > 1$. If y is feasible, then

$$g(y) - g(x) = f_1^k(y) - (f_1^k(x) + r(f_2^k(x) - \varepsilon)) = r(\varepsilon - f_2^k(x)) - 1$$

$$\geq r(\varepsilon - \lceil \varepsilon \rceil + 1) - 1 > 0,$$

where the first inequality uses $f_2^k(x) \leq \lceil \varepsilon \rceil - 1$ as x is infeasible and $f_2^k(x)$ is an integer, and the last inequality uses $r > \frac{1}{\varepsilon+1-\lceil \varepsilon \rceil}$ and $\varepsilon - \lceil \varepsilon \rceil + 1 > 0$. Hence, y will survive to the next generation. $\square$

Lemma 6. Consider using the randomized local search algorithm to maximize g defined as in (6) with $r > \frac{1}{\varepsilon+1-\lceil \varepsilon \rceil}$. Then once a feasible solution is reached, only the offspring via flipping one 0 bit in the last k positions (conditional on that the current solution has its f_2^k value at least $\lceil \varepsilon \rceil + 1$), or via flipping one 0 bit in the first $n - k$ positions will survive to the next population. Besides, all solutions in future generations are feasible.

Proof. Let x be a feasible solution in the current population and y, j, I_0, I_1, I'_0 and I'_1 be defined as in the proof of Lemma 5. If $j \in I_0$, then $f_1^k(y) > f_1^k(x)$ and $f_2^k(y) > f_2^k(x) \geq \varepsilon$. Thus y is feasible and $g(y) > g(x)$, and then this feasible y will survive to the next generation.

If $j \in I_1$, then $f_1^k(y) < f_1^k(x)$ and $f_2^k(y) < f_2^k(x)$. Thus, $g(y) < g(x)$ and y will be removed.

If $j \in I'_0$, then $f_1^k(y) = f_1^k(x) + 1$ and $f_2^k(y) = f_2^k(x) - 1$. If y is feasible, then $f_2^k(x) \geq \lceil \varepsilon \rceil + 1$, $g(y) = g(x) + 1$, and y will survive to the next generation. If y is infeasible, that is, $f_2^k(y) < \varepsilon$, then $f_2^k(y) \leq \lceil \varepsilon \rceil - 1$ as $f_2^k(y)$ is an integer. In this case,

$$g(y) - g(x) = f_1^k(y) + r(f_2^k(y) - \varepsilon) - f_1^k(x) \leq 1 + r(\lceil \varepsilon \rceil - 1 - \varepsilon) < 0,$$

where the last inequality uses $r > \frac{1}{\varepsilon+1-\lceil \varepsilon \rceil}$ and $\lceil \varepsilon \rceil - 1 - \varepsilon < 0$. Hence, such y will be removed in the selection.

If $j \in I'_1$, then $f_1^k(y) = f_1^k(x) - 1$ and $f_2^k(y) = f_2^k(x) + 1 > \varepsilon$. Then y is feasible and $g(y) = f_1^k(y) < f_1^k(x) = g(x)$. Hence, such y will be removed in the selection. $\square$

Now we state the runtime of RLS to maximize the constructed problem (6).

Theorem 2. Consider using the randomized local search algorithm to maximize g defined as in (6) with $r > \frac{1}{\varepsilon+1-\lceil \varepsilon \rceil}$. Then after $O(n \ln n)$ iterations in expectation, an optimal solution is reached.

Proof. We first consider the case for $\varepsilon \in (n - k, n]$.

We reuse the notations of I_0, I_1, I'_0 , and I'_1 in the proof of Lemma 5. Recalling Lemma 5, we know that before a feasible solution is reached for the first time, only the offspring via flipping one bit in $I_0 \cup I'_1$ will be accepted into the next population. Note that for any x , $|I_0 \cup I'_1| = n - f_2^k(x)$. Hence, a solution is infeasible iff $|I_0 \cup I'_1| \in [n - \lceil \varepsilon \rceil + 1, n]$ as an infeasible solution x has $f_2^k(x) \in [0, \lceil \varepsilon \rceil - 1]$. Therefore, the expected number of iterations to reach a feasible solution for the first time is at most

$$\sum_{i=n-\lceil \varepsilon \rceil+1}^n \frac{n}{i} \leq n(\ln n + 1). \quad (16)$$

Now we consider the case after a feasible solution is reached for the first time. For a feasible solution, we know that $|I_1| + |I'_0| \geq \varepsilon$ and further $|I_1| + |I'_0| \geq \lceil \varepsilon \rceil$ as $|I_1| + |I'_0| \in \mathbb{N}$. Then from $|I_1| + |I_0| = n - k$, we have

$$|I'_0| \geq \lceil \varepsilon \rceil - |I_1| = \lceil \varepsilon \rceil - n + k + |I_0| \geq \lceil \varepsilon \rceil - n + k. \quad (17)$$

Recalling Lemma 6, we know that only the offspring via flipping one bit in I_0 or via flipping one bit in I'_0 (conditional on the event, denoted as A , that the parent has f_2^k value of at least $\lceil \varepsilon \rceil + 1$) will be accepted into the next population, and all solutions in the future generations are feasible. Hence, in all future generations, both $|I_0|$ and $|I'_0|$ will not increase, the replacement can still happen if $|I_0| > 0$, and (17) always holds. Therefore, $|I_0 \cup I'_0| = \lceil \varepsilon \rceil - n + k$ happens iff $(|I_0| = 0, |I'_0| = \lceil \varepsilon \rceil - n + k)$, which is only reached by the optimal set $D_{\lceil \varepsilon \rceil} = \{x \mid f(x) = (2n - k - \lceil \varepsilon \rceil, \lceil \varepsilon \rceil)\}$. Hence, to reach an optimum, we need to decrease $|I_0 \cup I'_0|$ to $\lceil \varepsilon \rceil - n + k$. From $|I'_0| \geq \lceil \varepsilon \rceil - n + k + |I_0|$ in (17), we have

$$\begin{aligned} |I_0| + |I'_0| &\leq n - k + |I'_0| - \lceil \varepsilon \rceil + |I'_0| = n - k - \lceil \varepsilon \rceil + 2|I'_0| \\ &\leq n - k - \lceil \varepsilon \rceil + 2k = n + k - \lceil \varepsilon \rceil, \end{aligned}$$

where the last inequality uses $|I'_0| \leq k$. Hence, the expected number of iterations to reach an optimal solution for the first time is at most

$$\sum_{j=\lceil \varepsilon \rceil - n + k + 1}^{n+k-\lceil \varepsilon \rceil} \frac{n}{j} \leq n(\ln(n + k - \lceil \varepsilon \rceil) + 1). \quad (18)$$

Together with (16), we have $O(n \ln n)$ expected number of iterations for reaching an optimum for $\varepsilon \in (n - k, n]$.

Now we consider the case for $\varepsilon > n$. Recalling Lemma 4, we know that the optimal solution is $1^{n-k}0^k$, that is, when $|I_0| = |I'_1| = 0$. Note that all solutions are infeasible as $f_2^k \leq n$, and recall that (from Lemma 5) only the offspring that decreases $|I_0|$ or $|I'_1|$ will be accepted into the next population. Thus the expected number of iterations to reach the optimal solution $1^{n-k}0^k$ is at most

$$\sum_{i=1}^n \frac{n}{i} \leq n(\ln n + 1),$$

where we pessimistically consider $|I_0 \cup I'_1|$ decreasing from n to 0.

Now it remains to discuss the case for $\varepsilon \leq n - k$. Recalling Lemma 4, we know that the optimal solution is 1^n , that is, when $|I_0| = |I'_0| = 0$. If the initial solution is infeasible, then (16) shows that at most $n(\ln n + 1)$ expected iterations are needed to reach

a feasible solution for the first time. Thus, now we only consider that we start from a feasible solution. With additional restriction $|I_0| + |I'_0| \leq n$, analogous to (18), we know that the additional expected number of iterations to reach an optimal solution is at most

$$\sum_{i=1}^{\min\{n, n+k-\lceil \varepsilon \rceil\}} \frac{n}{i} \leq n(\ln n + 1).$$

Therefore, this theorem is proved. $\square$

Since there are $k+1$ Pareto front points, with Corollary 3 and Theorem 2, we have the following corollary to show the total runtime to cover the full Pareto front for ONEMAXMIN_k when ε and r are carefully chosen.

Corollary 4. Let $\varepsilon_0, \dots, \varepsilon_k$ be defined as in Corollary 3, $G = \{\varepsilon_0, \dots, \varepsilon_k\}$, and $r > \max_{i=1, \dots, k} \frac{1}{\varepsilon_i + 1 - \lceil \varepsilon_i \rceil}$. Consider using the randomized local search algorithm to maximize g with $\varepsilon \in G$ one by one (which terminates when the maximum of corresponding g is reached for the first time), and let P be set of all obtained solutions. Then within expected $O(\max\{k, 1\}n \ln n)$ fitness evaluations, P will have its size of $k+1$ and cover the full Pareto front.

5.3. Nonparameter penalty function

Another way for handling the constrained problem (5) is by nonparameter penalty function [33,34]. Instead of introducing a parameter to penalize the violated constraint in the exterior way discussed before, this nonparameter way adds the amount of violence directly to the objective. Formally, the nonparameter penalty way transfers the constrained problem (5) into maximizing the following function.

Definition 3. The nonparameter penalty function w.r.t. the constrained problem (5) $g : \{0, 1\}^n \rightarrow \mathbb{R}$ is defined by

$$g(x) = \begin{cases} f_1^k(x), & \text{if } x \text{ is feasible} \\ f_1^k(x) + f_2^k(x) - \varepsilon, & \text{if } x \text{ is infeasible} \end{cases} \quad (19)$$

for $x \in \{0, 1\}^n$.

It is not difficult to see that the above (19) is identical to (6) with $r = 1$. Recalling Lemma 4, we know that the optimal set for maximizing (19) is

$$\begin{cases} \bigcup_{i=n-k}^n D_i, & \varepsilon \geq n \\ \bigcup_{i=n-k}^{\lceil \varepsilon \rceil - 1} D_i, & \varepsilon \notin \mathbb{N}, \varepsilon \in (n-k, n) \\ \bigcup_{i=n-k}^{\lceil \varepsilon \rceil} D_i, & \varepsilon \in \mathbb{N}, \varepsilon \in (n-k, n) \\ D_{n-k}, & \varepsilon \leq n-k. \end{cases}$$

As discussed in Section 5.1, the optimal set of (5) is $\emptyset$ for $\varepsilon > n$, $D_{\lceil \varepsilon \rceil}$ for $\varepsilon \in (n-k, n]$, and D_{n-k} for $\varepsilon \leq n-k$. Hence, it is different from the optimal set of the problem (5), especially for $\varepsilon \in (n-k, n]$. For the setting that one ε value corresponds to one optimum solved by one algorithm, a specific $\varepsilon \in (n-k, n) \setminus \mathbb{N}$ will return a solution in D_j with the random variable $j \in [n-k, \lceil \varepsilon \rceil - 1]$ that follows an unknown distribution depending on the algorithm to solve (also for $\varepsilon \geq n$ and $\varepsilon \in (n-k, n) \cap \mathbb{N}$). Thus, the optimal solution set, say U , of (19) with different values of ε does not naturally ensure that $U \cap D_i \neq \emptyset$ holds for all $i = n-k, \dots, n$. That is, it does not naturally guarantee the full coverage of the Pareto front.

5.4. Summary and comment

From this section, we see that although the ε -constraint approach can cover the full Pareto front for ONEMAXMIN_k, the solving modes have difficulties or inconveniences. With careful settings of ε and penalty coefficient r , the exterior penalty function way can cover the full Pareto front with expected $O(\max\{k, 1\}n \ln n)$ function evaluations. However, the nonparameter penalty function way cannot guarantee that the optimal solution sets cover the full Pareto front.

We will also note that the above analysis shows that careful penalty design of solving modes is needed for constrained optimization.

6. Multiobjective evolutionary algorithms

In this section, we will show that the multiobjective evolutionary algorithms can easily cover the full Pareto front compared with the difficulty or the inconvenience witnessed in the typical approaches in Section 4.

6.1. MOEA/D

The MOEA/D is one kind of MOEAs that decomposes the multiobjective problem into several single-objective problems and solves them in a co-evolutionary way. Thus, it shares a similarity to the methods in Section 4 where also several single-objective problems are considered. Here, we first give a brief introduction of the MOEA/D and then discuss its runtime complexity.

Algorithm 1 MOEA/D for maximizing the bi-objective function $f = (f_1, f_2)$.

Parameters: $H + 1$: the number of the subproblems. $w_0, \dots, w_H$: the weight for the subproblems. T : the number of the subproblems to construct the neighbor of a given subproblem.

```

1: Construct  $h_i$  subproblem according to  $w_i$  for all  $i = 0, \dots, H$ , and calculate  $B_i = \{i_1, \dots, i_T\}$  such that  $w_{i_1}, \dots, w_{i_T}$  are closest weight vectors to  $w_i$ .
2: Initialize the external population  $P_e = \emptyset$ . Initialize the population  $P = \{x_0, \dots, x_H\}$ , evaluate  $f(x_i)$  and  $h_i(x_i)$  for  $i = 0, \dots, H$ , and set the reference point  $z^* = (\max\{f_1(z) \mid z \in P\}, \max\{f_2(z) \mid z \in P\})$ . For  $i \in [0..H]$ , let  $P_e = \{z \in P_e \mid z \not\leq x_i\} \cup \{x_i\}$  if  $x_i$  is not dominated by any solution in  $P_e$ .
3: for  $g = 1, 2, \dots$  do
4:   for  $i = 0, \dots, H$  do
5:     Generate  $x'_i$  via applying the mutation to  $x_i$ .
6:     For each  $j = 1, 2$ , if  $f_j(x'_i) > z_j^*$  then  $z_j^* = f_j(x'_i)$ .
7:     For each  $j \in B_i$  such that  $h_j(x'_i) \leq h_j(x_j)$ , set  $x_j = x'_i$ .
8:     If there is no  $z \in P_e$  such that  $x'_i < z$  then  $P_e = \{z \in P_e \mid z \not\leq x'_i\} \cup \{x'_i\}$ .
9:   end for
10: end for

```

6.1.1. Algorithm description

The MOEA/D is first proposed by [35] and the first theoretical runtime results are obtained by [8]. It first employs a decomposition method to decompose the multiobjective problem into the optimization of $H + 1$ subproblems. The Tchebycheff decomposition approach used in [35] constructs the i th ($i = 0, \dots, H$) subproblem (to be minimized) $h_i(x)$ with a predefined evenly spread weight w_i by

$$h_i(x) = \max\{w_i|f_1^k(x) - z_1^*|, (1 - w_i)|f_2^k(x) - z_2^*|\},$$

where z_i^* in the reference point $z^* = (z_1^*, z_2^*)$ is usually set to be the best f_i value found so far by the algorithm.

After the decomposition, the algorithm initializes the population, sets $z^* = (z_1^*, z_2^*)$ with z_i^* the best value of the i th objective, and puts the mutually incomparable solutions into the external population P_e . For each subproblem, one offspring is generated and z^* is updated accordingly. In its co-evolutionary solving mode, with predefined neighborhood size T , the i th subproblem has T subproblem neighbors with indices $\{i_1, \dots, i_T\} := B_i$ such that $w_{i_1}, \dots, w_{i_T}$ are T closest weight vectors to w_i . Then for each individual with the index of its corresponding subproblem in B_i , the individual itself and its fitness will be replaced by the generated offspring if this offspring has a better or the same h_i value. Later P_e is updated. The whole procedure is shown in Algorithm 1. As in the first theoretical work [8], we only set $T = 1$ and only consider generating the offspring by applying mutation to the i th individual in the population for the i th subproblem. As in [8], we expect that an optimal (minimal) value of a subproblem corresponds to a Pareto front point of ONEMAXMIN $_k$. Since the Pareto front size is $k + 1$, we then set $H = k$ and consider the weights $w_i = i/k, i = 0, \dots, H$. Note that in each iteration, z^* is updated before the fitness evaluation of the newly generated offspring. Hence, we easily know that before $1^{n-k}0^k$ (1^n) is reached, the minimization of the 0th (H th) subproblem is identical to the maximization of $f_2^k(x)$ ($f_1^k(x)$) via the $(1 + 1)$ EA. We also easily know that after both $1^{n-k}0^k$ and 1^n are reached, the i th subproblem $h_i(x)$ is formulated as

$$\begin{aligned} h_i(x) &= \max\{w_i|f_1^k(x) - n|, (1 - w_i)|f_2^k(x) - n|\} \\ &= \max\left\{\frac{i}{k}\left(n - \sum_{j=1}^n x_j\right), \frac{k-i}{k}\left(n - \sum_{j=1}^{n-k} x_j - \sum_{j=n-k+1}^n (1 - x_j)\right)\right\}. \end{aligned} \quad (20)$$

6.1.2. Runtime analysis

Now we analyze the runtime of the MOEA/D to cover the full Pareto front. We first have the following lemma for the set of the optimal solutions for minimizing $h_i(x)$.

Lemma 7. Let $H = k$ and $S_i = \{x \mid f^k(x) = (n - k + i, n - i)\}$ for $i = 1, \dots, H - 1$. Then for $h_i(x)$ defined in (20), let $h_i^1(x) = \frac{i}{k}(n - f_1^k(x)) = \frac{i}{k}(n - \sum_{j=1}^n x_j)$ and $h_i^2(x) = \frac{k-i}{k}(n - f_2^k(x)) = \frac{k-i}{k}(n - \sum_{j=1}^{n-k} x_j - \sum_{j=n-k+1}^n (1 - x_j))$, then we have

- if $(n - |x|_1) > k - i, |x_{[1..n-k]}|_1 = n - k$ or $x \in S_i$, then $h_i(x) = h_i^1(x)$;
- if $n - |x|_1 \leq k - i$, then $h_i(x) = h_i^2(x)$;
- $\arg \min_{x \in \{0,1\}^n} h_i(x) = S_i$.

Proof. Consider any $x \in \{0, 1\}^n$.

We first consider the case when $n - |x|_1 = k - i$, that is, when $f_1^k(x) = n - k + i$. In this case, $h_i^1(x) = (i(k - i))/k$. If $x \in S_i$, that is,

$$f_2^k(x) = \left(\sum_{j=1}^{n-k} x_j\right) + \left(\sum_{j=n-k+1}^n 1 - x_j\right) = n - i,$$

then $h_i^2(x) = \frac{k-i}{k}(n - (n-i)) = (i(k-i))/k$. Hence,

$$h_i(x) = h_i^1(x) = h_i^2(x) = \frac{i(k-i)}{k}. \quad (21)$$

If $x \notin S_i$, it is not difficult to see that $|x_{[1..n-k]}|_1 < n-k$ and thus $|x_{[n-k+1..n]}|_1 > i$ from $f_1^k(x) = n-k+i$. Then

$$f_2^k(x) = |x_{[1..n-k]}|_1 + k - |x_{[n-k+1..n]}|_1 < n-k+k-i = n-i,$$

thus

$$h_i^2(x) > \frac{k-i}{k}(n - (n-i)) = \frac{i(k-i)}{k} = h_i^1(x).$$

Hence,

$$h_i(x) = h_i^2(x) > h_i^1(x) = \frac{i(k-i)}{k}. \quad (22)$$

If $n - |x|_1 > k-i$, then

$$h_i(x) \geq \frac{i}{k} \left(n - \sum_{j=1}^n x_j \right) > \frac{i}{k}(k-i) = \frac{i(k-i)}{k}. \quad (23)$$

If further $|x_{[1..n-k]}|_1 = n-k$, then $|x_{[n-k+1..n]}|_1 < i$ from $n - \sum_{j=1}^n x_j > k-i$. Hence, $f_2^k(x) > n-k+k-i = n-i$. Then $h_i^2(x) < \frac{(k-i)i}{k} < h_i^1(x)$, and thus $h_i(x) = h_i^1(x)$, proving the first property.

If $n - |x|_1 < k-i$, we have $h_i^1(x) < (i(k-i))/k$, and $\sum_{j=1}^n (1-x_j) < k-i$. Then

$$\begin{aligned} h_i^2(x) &= \frac{k-i}{k} \left(n - \sum_{j=1}^{n-k} x_j - \sum_{j=n-k+1}^n (1-x_j) \right) \\ &\geq \frac{k-i}{k} \left(n - \sum_{j=1}^{n-k} x_j - \sum_{j=1}^n (1-x_j) \right) > \frac{k-i}{k} \left(n - \sum_{j=1}^{n-k} x_j - (k-i) \right) \\ &\geq \frac{k-i}{k}(n - (n-k) - (k-i)) = \frac{i(k-i)}{k}. \end{aligned}$$

Hence,

$$h_i(x) = h_i^2(x) \geq \frac{i(k-i)}{k}. \quad (24)$$

Thus, with (21) and (22), the second property is proved.

From (21) to (24) and noting (21) is for $x \in S_i$, the last property is proven. We also note here that we do not consider the case $(n - |x|_1 > k-i, |x_{[1..n-k]}|_1 \neq n-k)$ as it is not needed for our proof in Theorem 3. $\square$

We note that the optimal h_0 and h_H are $1^{n-k}0^k$ and 1^n respectively. As discussed before, the minimization of h_0 (h_H) is identical to the maximization of $f_2^k(x)$ ($f_1^k(x)$) via the $(1+1)$ EA. Then after $O(n \log n)$ generations, both points will be found. Then the subproblems to solve will be (20). Lemma 7 shows that the optimal set of each subproblem is corresponding to one Pareto front point and different subproblem is related to different Pareto front points. Thus solving all subproblems will result in a full Pareto front coverage. Since $T = 1$, we only need to calculate the runtime for each subproblem, which is the same as the one that RLS is used to solve this subproblem, and then summing them up results in the final runtime. Hence, we have the following theorem.

Theorem 3. Let $H = k$ and $w_i = i/k, i = 0, \dots, H$. The expected runtime for the MOEA/D applying one-bit mutation to optimize ONEMAXMIN_k is $O(\max\{k, 1\}n \ln n)$.

Proof sketch. As discussed above, we only need to consider the runtime of the RLS to solve the i th subproblem for $i = 1, \dots, H-1$. We divide the process into two phases. The first phase ends when an x with $|x_{[1..n-k]}|_1 = n-k$ is reached for the first time, and the second phase starts right after the end of the first phase, and ends when an optimum of $h_i(x)$ is reached for the first time.

We note that in the first phase, flipping any zeros in $[1..n-k]$ positions will decrease $h_i(x)$ and flipping any ones in $[1..n-k]$ positions will increase $h_i(x)$. Also note that $[1..n-k]$ bits will not change if the flip happens in $[n-k+1..n]$ positions as we use one-bit mutation. Hence, after at most $O(n \ln(n-k))$ iterations, an x with $|x_{[1..n-k]}|_1 = n-k$ will be reached for the first time. In the second phase, for the one-bit mutation, it is not difficult to see that flipping any ones in $[1..n-k]$ positions will increase $h_i(x)$ as well. Hence, for the minimization of $h_i(x)$, we know that all individuals in the future generations will keep $|x_{[1..n-k]}|_1 = n-k$. Then from Lemma 7, we know that if the second phase starts with $n - |x|_1 > k-i$, then the whole second phase will minimize $h_i^1(x)$, and if it starts with $n - |x|_1 \leq k-i$, then the whole second phase will minimize $h_i^2(x)$. Similar to the ONEMAX optimization, the upper bound for the second phase is $O(n \ln k)$.

Considering all subproblems proves this theorem. $\square$

6.2. MOEAs without decomposition

The above subsection discussed the runtime for the MOEAs with decomposition. The most widely used MOEA is the NSGA-II, which does not rely on decomposing the multiobjective problems into several single-objective problems. In this section we will discuss the runtime for the MOEAs without decomposition.

Algorithm 2 (G)SEMO and SMS-EMOA with population size μ for maximizing the multiobjective function f .

- 1: For (G)SEMO: Generate $x \in \{0, 1\}^n$ uniformly at random and $P \leftarrow \{x\}$; For SMS-EMOA: Generate μ individuals uniformly at random from $\{0, 1\}^n$ and form P .
 - 2: **for** $g = 1, 2, \dots$ **do**
 - 3: Select x from P uniformly at random
 - 4: Generate x' via applying mutation operator (one-bit mutation for SEMO and standard bit-wise mutation for the GSEMO and SMS-EMOA) on x
 - 5: $P \leftarrow \text{SurvivalSelection}(P, x')$
 - 6: **end for**
-

Algorithm 3 SurvivalSelection(P, x').

- 1: **if** (G)SEMO **then**
 - 2: **if** there exists no $y \in P$ such that $x' < y$ **then**
 - 3: $P \leftarrow \{z \in P \mid z \not\leq x'\} \cup \{x'\}$
 - 4: **else**
 - 5: $P \leftarrow P$
 - 6: **end if**
 - 7: **else if** SMS-EMOA **then**
 - 8: Use fast-non-dominated-sort() in [36] to divide $P \cup \{x'\}$ into fronts $F_1, F_2, \dots, F_d$ for some positive integer d
 - 9: Uniformly at random choose y from $\arg \min_{z \in F_d} \text{HV}(F_d) - \text{HV}(F_d \setminus \{z\})$
 - 10: $P \leftarrow F_d \setminus \{y\}$
 - 11: **end if**
-

6.2.1. Algorithm descriptions

(G)SEMO is the basic toy algorithm analyzed in the theory community. It starts with a single randomly generated individual. In each generation, one individual is uniformly at random selected as parent to generate one offspring via one-bit or standard bit-wise mutation. If this offspring cannot be dominated by any individual in the current population, it will kick out all individuals that are weakly dominated by it, and enter into the next generation. See details in Algorithm 2.

Different from (G)SEMO, the NSGA-II [36] works with a fixed size N of population. In each generation, N offspring individuals are generated, and it uses non-dominated sorting and crowding distance to remove the worst N individuals in the combined parent and offspring population R_t . The non-dominated sorting procedure will first divide R_t into several front F_i , with F_i the non-dominated solutions in $R_t \setminus \bigcup_{j=0}^{i-1} F_j$. For the critical front F_{i^*} with $i^* = \min\{j \mid |\bigcup_{i=0}^j F_i| \geq N\}$, the algorithm calculates the crowding distances of all individuals in F_{i^*} and keep the ones with $N - |\bigcup_{j=0}^{i^*-1} F_j|$ largest crowding distance (tie broken at random). See details in Algorithm 4. It is the most widely used MOEAs and the first runtime is conducted in [3,4]. A series of theoretical analyses on the NSGA-II are conducted later, like [10,24,25,37–44].

Algorithm 4 NSGA-II with population size N for maximizing the multiobjective function f .

- 1: Uniformly at random generate the initial population $P_0 = \{x_1, x_2, \dots, x_N\}$ with $x_i \in \{0, 1\}^n, i = 1, 2, \dots, N$.
 - 2: **for** $t = 0, 1, 2, \dots$ **do**
 - 3: Generate the offspring population Q_t with size N
 - 4: Use fast-non-dominated-sort() in [36] to divide $R_t = P_t \cup Q_t$ into $F_1, F_2, \dots$
 - 5: Find $i^* \geq 1$ such that $\sum_{i=1}^{i^*-1} |F_i| < N$ and $\sum_{i=1}^{i^*} |F_i| \geq N$
 - 6: Use Algorithm 5 to separately calculate the crowding distance of each individual in $F_1, \dots, F_{i^*}$
 - 7: Let $\tilde{F}_{i^*}$ be the $N - \sum_{i=0}^{i^*-1} |F_i|$ individuals in F_{i^*} with largest crowding distance, chosen at random in case of a tie
 - 8: $P_{t+1} = \left(\bigcup_{i=1}^{i^*-1} F_i\right) \cup \tilde{F}_{i^*}$
 - 9: **end for**
-

The SMS-EMOA [45] is a variant of the steady-state (in a $(\mu + 1)$ scheme) NSGA-II, which replaces the crowding distance by the hypervolume contribution where the hypervolume is defined in the following. Note that in the rest of the paper, one could set the reference point r strictly less than all possible function values, like $r = (-1, -1)$, and we will not explicitly state its value in our theoretical results.

Definition 4 (Hypervolume). Let a reference point r be dominated by all Pareto optimal solutions. The *hypervolume* of a set S of individuals w.r.t. r in the objective space is defined as

$$\text{HV}_r(S) = \mathcal{L}\left(\bigcup_{u \in S} \{h \in \mathbb{R}^m \mid r \leq h \leq f(u)\}\right),$$

where $\mathcal{L}$ is the Lebesgue measure, and for $u, v \in \mathbb{R}^m$, $u \leq v$ is defined by $u_i \leq v_i$ for all $i \in [1..m]$.

Algorithm 5 crowding-distance(S).**Input:** $S = \{S_1, \dots, S_{|S|}\}$: the set of individuals**Output:** $\text{cDis}(S) = (\text{cDis}(S_1), \dots, \text{cDis}(S_{|S|}))$, the vector of crowding distances of the individuals in S

```

1:  $\text{cDis}(S) = (0, \dots, 0)$ 
2: for each objective function  $f_i$  do
3:   Sort  $S$  in order of ascending  $f_i$  value:  $S_{i,1}, \dots, S_{i,|S|}$ 
4:    $\text{cDis}(S_{i,1}) = +\infty, \text{cDis}(S_{i,|S|}) = +\infty$ 
5:   for  $j = 2, \dots, |S| - 1$  do
6:      $\text{cDis}(S_{i,j}) = \text{cDis}(S_{i,j}) + \frac{f_i(S_{i,j+1}) - f_i(S_{i,j-1})}{f_i(S_{i,|S|}) - f_i(S_{i,1})}$ 
7:   end for
8: end for

```

The hypervolume contribution of an individual is the hypervolume difference between the cases of including this individual or not. The individual with the smallest hypervolume contribution will be removed. See details in Algorithm 2. The first runtime analysis of the SMS-EMOA is conducted by Bian et al. [7] for the bi-objective OJZJ, and a stochastic population update strategy is proposed. Later, its runtime on DLTB is conducted by Zheng et al. [15]. Zheng and Doerr [46] proved the first runtime of the SMS-EMOA for many objectives, and showed that it performs well compared with the inefficiency of the NSGA-II. Besides, they also proved that the speed-up witnessed by the stochastic population in [7] for two objectives has a reduced impact when the number of objectives grows, but proved that the heavy-tailed mutation has a good runtime speed-up.

6.2.2. Runtime analyses

Now we discuss the runtime of the (G)SEMO, the NSGA-II, and the SMS-EMOA. As mentioned before, Antipov, Kötzing, and Radhakrishnan [9] proved the expected runtime of $O(kn \ln n)$ for (G)SEMO for an essentially identical problem of minimizing $(\text{OM}_a, \text{OM}_b)$ with $k = H(a, b)$. Here, for completeness, we also include the runtime of the (G)SEMO in our analysis as its similar proof procedure to that for the NSGA-II and SMS-EMOA. The following lemma estimates the probability of selecting an individual as a parent in one iteration for any algorithm that generates the offspring population with the same size as the parent population P . We consider the following three ways. The *fair selection* will let each individual in P be selected as a parent once. In the *random selection*, we $|P|$ times independently and uniformly at random pick a parent from P . The *binary tournament selection* generates $|P|$ parents by independently conducting $|P|$ -round competitions (with replacement). In each round, two individuals in $|P|$ are uniformly at random selected, and the better (e.g. based on the non-dominated sorting and crowding distance for the NSGA-II) one will be picked as a parent.

Lemma 8. For a population P with size N , we have the following results.

- For any $x \in P$, the probability of having x selected as a parent in an iteration is 1 for fair selection, and at least $1 - 1/e$ for random selection.
- Let $m, n \in \mathbb{N}_+$ and $m = o(N)$. Let F'_1 be the set of non-dominated solutions (within P) w.r.t. some m -objective f . Let $f_1^{\max} = \max\{f_1(x) \mid x \in F'_1\}$, $f_i^{\max} = \max\{f_i(x) \mid x \in F'_1, f_j(x) = f_j^{\max}, j = 1, \dots, i-1\}$ for $i = 2, \dots, m$, and $X = \{x \in F'_1 \mid f(x) = (f_1^{\max}, \dots, f_m^{\max})\}$. Then for the NSGA-II there is an individual $\tilde{x} \in X$ with $\text{cDis}(\tilde{x}) = +\infty$, and the probability that $\tilde{x}$ is selected as a parent in an iteration is $\Omega(1)$ for binary tournament selection.

Proof. Since each individual in P will be selected as a parent for fair selection, the probability of selecting x as a parent is 1. For random selection, such probability is

$$1 - \left(\frac{N-1}{N}\right)^N \geq 1 - \frac{1}{e}.$$

Now we consider the binary tournament selection. Since $X \subseteq F'_1$ and X contains individuals with the largest f_1 value, then there is an $\tilde{x} \in X$ with $\text{cDis}(\tilde{x}) = +\infty$ from Algorithm 5. Hence, $\tilde{x}$ will win in the competition against other individuals with a finite crowding distance. Note that there are at most $2m$ individuals with infinite crowding distances. Hence, in each round of comparison, the probability of choosing $\tilde{x}$ and one with finite crowding distance is at least

$$\frac{\binom{N-2m}{1}}{\binom{N}{2}} = \frac{2(N-2m)}{N(N-1)}.$$

Then the probability of selecting $\tilde{x}$ as a parent is at least

$$\begin{aligned}
1 - \left(1 - \frac{2(N-2m)}{N(N-1)}\right)^N &= 1 - \left(1 - \frac{2(N-2m)}{N(N-1)}\right)^{\frac{N(N-1)}{2(N-2m)} \cdot \frac{2(N-2m)}{N-1}} \\
&\geq 1 - \exp\left(-\frac{2(N-2m)}{N-1}\right). \quad \square
\end{aligned}$$

The following lemma shows that once a Pareto front point is reached, all future generations will contain at least one individual with its function value as such a Pareto front. The result for the (G)SEMO is trivial, the one for the SMS-EMOA is a corollary of [46, Lemma 4], and the result for the NSGA-II can be easily obtained from [4, Proof of Lemma 1] that each function value will have at most four corresponding individuals with positive crowding distance values.

Lemma 9 ([4,46]). *Consider using (G)SEMO, SMS-EMOA with $\mu \geq k + 1$, or the NSGA-II with $N \geq 4(k + 1)$ to optimize the ONEMAXMIN_k problem. At some time t_0 , let x be a solution that exists in the current population P_{t_0} or is one offspring of P_{t_0} . Then for any $t > t_0$ there is a y in the population P_t such that $y \succeq x$. Particularly, if x is Pareto optimal, then there is a $y \in P_t$ for any $t > t_0$ such that $f^k(y) = f^k(x)$.*

The following lemma estimates the runtime to reach at least one Pareto optimal solution. Note here that in the following, all theoretical results hold for the NSGA-II with standard bit-wise mutation or one-bit mutation.

Lemma 10. *Consider using (G)SEMO, SMS-EMOA with $\mu \geq k + 1$, or the NSGA-II with $N \geq 4(k + 1)$ to optimize the ONEMAXMIN_k problem. After $O(\max\{k, 1\}n \ln n)$ ((G)SEMO), $O(\mu n \ln n)$ (SMS-EMOA), and $O(nN \ln n)$ (NSGA-II) number of function evaluations in expectation, the population (and all populations afterwards) will contain at least one Pareto optimal solution.*

Proof. We consider the runtime to reach a Pareto optimal solution for the first time. More specifically, we consider the first time to reach 1^n . Now we reuse notations in Lemma 8. For the current population P , let $\eta = \max_{x \in P} f_1(x)$. Then it is not difficult to see that $\eta = f_1^{\max}$ and η will not decrease as the population evolves. Let $\tilde{x} \in X$ with infinite crowding distance. Note that the probability of selecting $\tilde{x}$ as a parent in an iteration is $\Omega(1)$ for the NSGA-II from Lemma 8, $\frac{1}{|P|} \geq \frac{1}{k+1}$ for the (G)SEMO from Lemma 2, and $\frac{1}{\mu}$ for the SMS-EMOA.

We note that the probability to generate an offspring x' with $f_1(x') > \eta$ from the chosen $\tilde{x}$ is at least

$$\frac{n - \eta}{n} \left(1 - \frac{1}{n}\right)^{n-1} \geq \frac{n - \eta}{en}$$

for the standard bitwise mutation, and it is $\frac{n - \eta}{n}$ for the one-bit mutation.

Now we show that if the offspring population Q contains an individual with f_1 value larger than η , then at least one individual y with $f_1(y) > \eta$ will survive to the next population for all algorithms. For (G)SEMO, $Q = \{y\}$ and y is not dominated by others, and thus y will survive. For SMS-EMOA, $Q = \{y\}$ and y is not dominated by others as well. Then $y \in F_1$ and its hypervolume contribution is positive. Note that the only removal will happen in the last front F_d for $d > 1$, or in F_1 with minimal hypervolume contribution for $d = 1$. For the former case, y will enter into the next generation as $y \in F_1$. For the latter case, since $|f_1(F_1)| \leq k + 1$ from Lemma 2 and $|P \cup Q| \geq N + 1 > k + 1$, there is an individual with hypervolume contribution as zero. One individual with zero hypervolume contribution will be removed and y will survive. For the NSGA-II, since Q contains an individual with f_1 value larger than η , we know that there is an individual y with the largest f_1 value and with infinite crowding distance. Obviously, $y \in F_1$. Note that each function value will have at most four corresponding individuals with positive crowding distance values. Since $|f(F_1)| \leq k + 1$ from Lemma 2, we know that there are at most $4(k + 1)$ individuals with positive crowding distance. With $N \geq 4(k + 1)$, we know that y will survive to the next population.

Since η does not decrease, we know that the expected iterations to increase η is at most $O\left(\frac{n}{n - \eta}\right)$ iterations for the NSGA-II, at most $O\left(\frac{(k+1)n}{n - \eta}\right)$ for the (G)SEMO, and at most $O\left(\frac{\mu n}{n - \eta}\right)$ for the SMS-EMOA. Hence, to reach 1^n , that is, $\eta = n$, we have the upper bound of expected iterations of

$$\sum_{\eta=0}^{n-1} O\left(\frac{n}{n - \eta}\right) = O(n \ln n)$$

for the NSGA-II,

$$\sum_{\eta=0}^{n-1} O\left(\frac{(k+1)n}{n - \eta}\right) = O(n \max\{k, 1\} \ln n)$$

for the (G)SEMO, and

$$\sum_{\eta=0}^{n-1} O\left(\frac{\mu n}{n - \eta}\right) = O(\mu n \ln n)$$

for the SMS-EMOA. Noting that N offspring solutions are generated and evaluated in each iteration for the NSGA-II, and one offspring solution for the (G)SEMO and SMS-EMOA, we proved this lemma. $\square$

The following lemma shows the runtime for the full coverage of the Pareto front after one Pareto front point is reached.

Lemma 11. *Consider using (G)SEMO, SMS-EMOA with $\mu \geq k + 1$, or the NSGA-II with $N \geq 4(k + 1)$ to optimize the ONEMAXMIN_k problem. Assume that the population contains a Pareto optimal solution. Then after $O(\max\{k, 1\}n \max\{\ln k, 1\})$ ((G)SEMO), $O(\mu n \max\{\ln k, 1\})$ (SMS-EMOA), and $O(nN \max\{\ln k, 1\})$ (NSGA-II) number of function evaluations in expectation, the population (and all populations afterward) will cover the whole Pareto front.*

Proof. For all $i \in [0..k-1]$, we call $(n-k+i, n-i)$, and $(n-k+i+1, n-i-1)$ are *neighbors*, and also $(n-k+i, n-i)$ is the *left neighbor* of $(n-k+i+1, n-i-1)$ and $(n-k+i+1, n-i-1)$ is the *right neighbor* of $(n-k+i, n-i)$. Let x be a Pareto optimum in the current population, satisfying that at least one neighbor of $f(x) = (n-k+j, n-j)$ is not reached. Then we know that x contains j ones and $k-j$ zeros in $x_{[n-k+1..n]}$. Hence, it is sufficient to flip exactly a one bit in $x_{[n-k+1..n]}$ (with other bits in x unchanged) to create a left neighbor, and exactly a zero bit in $x_{[n-k+1..n]}$ for a right neighbor. Therefore, if the left neighbor of $f(x)$ is not reached, then the probability to generate x' with $f(x') = (n-k+j-1, n-j+1)$ from x is j/n for the one-bit mutation, and at least

$$\frac{j}{n} \left(1 - \frac{1}{n}\right)^{n-1} \geq \frac{j}{en}$$

for the standard bitwise mutation. That is, such probability is $\Omega\left(\frac{j}{n}\right)$ for both mutation strategies. Similarly, if the right neighbor of $f(x)$ is not reached, then the probability to generate x' with $f(x') = (n-k+j+1, n-j-1)$ from x is $\frac{k-j}{n}$ for the one-bit mutation, and at least

$$\frac{k-j}{n} \left(1 - \frac{1}{n}\right)^{n-1} \geq \frac{k-j}{en}$$

for the bitwise mutation. That is, such probability is $\Omega\left(\frac{k-j}{n}\right)$ for both mutation strategies.

Together with the probability that x is chosen as a parent in one iteration from Lemma 8, we know that for the NSGA-II, the probability is $\Omega\left(\frac{j}{n}\right)$ to generate a left neighbor of x in one iteration and is $\Omega\left(\frac{k-j}{n}\right)$ to generate a right neighbor. For the (G)SEMO, the corresponding two probabilities are $\Omega\left(\frac{j}{n|P|}\right) = \Omega\left(\frac{j}{n(k+1)}\right)$ and $\Omega\left(\frac{k-j}{n|P|}\right) = \Omega\left(\frac{k-j}{n(k+1)}\right)$. For the SMS-EMOA, the corresponding two probabilities are $\Omega\left(\frac{j}{n\mu}\right)$ and $\Omega\left(\frac{k-j}{n\mu}\right)$.

From Lemma 9, we know that any reached Pareto front point will be maintained in all future populations. Hence, the expected number of iterations to cover the full Pareto front is

$$\sum_{j=1}^k O\left(\frac{n(k+1)}{j}\right) + \sum_{j=0}^{k-1} O\left(\frac{n(k+1)}{k-j}\right) = O(n(k+1) \max\{\ln k, 1\})$$

for the (G)SEMO,

$$\sum_{j=1}^k O\left(\frac{n\mu}{j}\right) + \sum_{j=0}^{k-1} O\left(\frac{n\mu}{k-j}\right) = O(n\mu \max\{\ln k, 1\})$$

for the SMS-EMOA, and

$$\sum_{j=1}^k O\left(\frac{n}{j}\right) + \sum_{j=0}^{k-1} O\left(\frac{n}{k-j}\right) = O(n \max\{\ln k, 1\})$$

for the NSGA-II. $\square$

Together with the above runtime results, we have the following runtime for the full coverage of the Pareto front.

Theorem 4. Consider using (G)SEMO, SMS-EMOA with $\mu \geq k+1$, or the NSGA-II with $N \geq 4(k+1)$ to optimize ONEMAXMIN_k problem. Then the expected number of function evaluations to cover the full Pareto front is $O(\max\{k, 1\}n \ln n)$ for (G)SEMO, $O(n\mu \ln n)$ for the SMS-EMOA, and $O(nN \ln n)$ for the NSGA-II.

6.3. Summary and comment

Recall in Section 5 that with careful settings of ε and penalty coefficient r in the exterior penalty function way, the ε -constraint approach can cover the full Pareto front in expected $O(\max\{k, 1\}n \ln n)$ function evaluations. Theorems 3 and 4 show that the generally analyzed MOEAs, the MOEA/D, the (G)SEMO, the NSGA-II, and the SMS-EMOA can easily cover the full Pareto front with the same asymptotic runtime.

Besides, as mentioned before, if we merely consider solving the constrained problem (5) as our goal, Theorem 2 in Section 5 shows that although the exterior penalty function way can solve it, a careful design of the penalty coefficient is required. Theorems 3 and 4 show the possibility of solving a constrained problem via MOEAs. That is, transfer a constrained problem to a multiobjective optimization problem, and then pick the best solution w.r.t. the original constrained problem among the Pareto set. This is an easy way to implement with the cost of more runtime, like in this example, $O(\max\{k, 1\}n \ln n)$ (Theorems 3 and 4) for the MOEAs but $O(n \ln n)$ (Theorem 2) for the RLS to solve the exterior penalty function with carefully chosen coefficient.

7. Brief discussion of a LEADINGONES variant

Our comparison of MOEAs and non-MOEA methods was built on a bi-objective ONEMAX variant. To address the question of whether these observations hold for other problems, this section will give brief discussions on performance for the variant of LEADINGONES, another popular benchmark analyzed in the evolutionary theory community. We will show that similar situations are witnessed for the non-MOEAs and MOEAs. As a side result, when there is no conflict in the two objectives, the discussed MOEAs reach the same runtime of $O(n^2)$ as many single-objective algorithms for the maximization of LEADINGONES.

7.1. LOTZ_k

As discussed in Section 2.2, the LOTZ benchmark is a popular bi-objective counterpart of the single-objective LEADINGONES benchmark. Indeed, we note that this benchmark can be regarded as a special case of the following benchmark with $k = n$.

Definition 5. Let $\ell \in \mathbb{N}$, and let $\text{LO}(y) = \sum_{i=1}^{\ell} \prod_{j=1}^i y_j$ and $\text{TZ}(y) = \sum_{i=1}^{\ell} \prod_{j=n-i+1}^{\ell} (1 - y_j)$ for any $y = (y_1, \dots, y_{\ell})$. The LOTZ_k function $\{0, 1\}^n \rightarrow \mathbb{R}^2$ is defined by

$$h^k(x) = (h_1^k(x), h_2^k(x)) = (\text{LO}(x), \text{LO}(x_{[1..n-k]}) + \text{TZ}(x_{[n-k+1..n]}))$$

for $x = (x_1, \dots, x_n) \in \{0, 1\}^n$.

Intuitively, the two objectives of LOTZ_k concur in the first $n - k$ bit positions but conflict in the last k bit positions. We call k the degree of conflict of this benchmark. The following two lemmas respectively give the Pareto front and the maximal number of non-dominated function values (the maximal number of incomparable solutions).

Lemma 12. Let M denote the Pareto front of LOTZ_k. Then

$$M = \{(n - k + i, n - i) \mid i \in [0..k]\},$$

and the Pareto set for LOTZ_k is $S^* := \{1^{n-k+i}0^{k-i} \mid i \in [0..k]\}$.

Proof. We first show that $y \not\succeq x$ for any $x \in S^*$ and for any $y \in \{0, 1\}^n$. If $y \succ x$, from the definition of dominance, we know

$$h_1^k(y) + h_2^k(y) > h_1^k(x) + h_2^k(x). \quad (25)$$

From Definition 5 we have

$$\begin{aligned} h_1^k(y) + h_2^k(y) &= 2\text{LO}(y_{[1..n-k]}) + \sum_{i=n-k+1}^n \prod_{j=1}^i y_j + \text{TZ}(y_{[n-k+1..n]}) \\ &\leq 2(n - k) + k = 2n - k = h_1^k(x) + h_2^k(x), \end{aligned}$$

which contradicts (25). Hence $y \not\succeq x$.

It remains to show that for any $y \in \{0, 1\}^n \setminus S^*$, there is an $x \in S^*$ such that $x \succ y$. Let $a = \text{LO}(y)$. If $a < n - k$, then let $x = 1^{n-k}0^k$. Then $x \in S^*$, $h_1^k(y) < n - k = h_1^k(x)$, and $h_2^k(y) < n = h_2^k(x)$. Thus $x \succ y$. If $a \geq n - k$, then let $x = 1^a 0^{n-a}$. Then $a < n$ and $x \in S^*$. Since $y \notin S^*$, we know $h_2^k(y) < n - k + n - a = h_2^k(x)$. Noting that $h_1^k(y) = a = h_1^k(x)$, we know that $x \succ y$. $\square$

Lemma 13. The maximal number of pairwise non-dominated function values w.r.t. LOTZ_k is $k + 1$.

Proof. Let V be the set of mutually incomparable solutions. We now show that for any incomparable $x, y \in V$, we have $\text{TZ}(x_{[n-k+1..n]}) \neq \text{TZ}(y_{[n-k+1..n]})$. If $\text{TZ}(x_{[n-k+1..n]}) = \text{TZ}(y_{[n-k+1..n]})$, from Definition 5 we have that

- if $\text{LO}(x_{[1..n-k]}) > \text{LO}(y_{[1..n-k]})$, then $h^k(x) > h^k(y)$;
- if $\text{LO}(x_{[1..n-k]}) < \text{LO}(y_{[1..n-k]})$, then $h^k(x) < h^k(y)$;
- if $\text{LO}(x_{[1..n-k]}) = \text{LO}(y_{[1..n-k]})$, then $h_2^k(x) = h_2^k(y)$,

any of which contradicts that x and y are incomparable. Since $\text{TZ}(x_{[n-k+1..n]}) \in [0..k]$, we have $|V| \leq k + 1$. With the Pareto front size of $k + 1$ from Lemma 12, this lemma is proven. $\square$

7.2. Difficulty of the scalarization approach

In this subsection, we will discuss the performance of the scalarization approach to maximize LOTZ_k. It will transfer the original problem into the maximization of the following function.

Definition 6. Let $w \in \mathbb{R}$. The scalarization function $h_w^k : \{0, 1\}^n \rightarrow \mathbb{R}$ of LOTZ_k is defined by

$$\begin{aligned} h_w^k(x) &= wh_1^k(x) + (1 - w)h_2^k(x) \\ &= \text{LO}(x_{[1..n-k]}) + w \sum_{i=n-k+1}^n \prod_{j=1}^i x_j + (1 - w)\text{TZ}(x_{[n-k+1..n]}), \end{aligned}$$

for $x = (x_1, \dots, x_n) \in \{0, 1\}^n$.

The following lemma calculates the optimal solution set w.r.t. the maximization of h_w^k .

Lemma 14. Let S^* be defined as in Lemma 12. Let $S_w = \{x \in \{0, 1\}^n \mid h_w^k(y) \leq h_w^k(x), \text{ for all } y \in \{0, 1\}^n\}$ be the set of optima of h_w^k . Then $S_w = \{1^{n-k}0^k\}$ for $w \in [0, 1/2)$, $S_w = \{1^n\}$ for $w \in (1/2, 1]$, and $S_w = S^*$ for $w = 1/2$.

Proof. If $w \in [0, 1/2)$, for any $y \neq 1^{n-k}0^k$, then either $\text{LO}(y_{[1..n-k]}) \neq n-k$ or $\text{TZ}(y_{[n-k+1..n]}) \neq k$. For $\text{LO}(y_{[1..n-k]}) \neq n-k$, we have $\text{LO}(y_{[1..n-k]}) < n-k$. With $\text{TZ}(y_{[n-k+1..n]}) \leq k$ and $w < 1/2$, we know

$$\begin{aligned} h_w^k(y) &= \text{LO}(y_{[1..n-k]}) + w \sum_{i=n-k+1}^n \prod_{j=1}^i y_j + (1-w)\text{TZ}(y_{[n-k+1..n]}) \\ &= \text{LO}(y_{[1..n-k]}) + (1-w)\text{TZ}(y_{[n-k+1..n]}) \\ &< n-k + (1-w)k = h_w^k(1^{n-k}0^k). \end{aligned} \quad (26)$$

For $\text{TZ}(y_{[n-k+1..n]}) \neq k$, we have $\text{TZ}(y_{[n-k+1..n]}) < k$. With $\text{LO}(y_{[1..n-k]}) \leq n-k$ and $w < 1/2$, we know

$$\begin{aligned} h_w^k(y) &= \text{LO}(y_{[1..n-k]}) + w \sum_{i=n-k+1}^n \prod_{j=1}^i y_j + (1-w)\text{TZ}(y_{[n-k+1..n]}) \\ &\leq \text{LO}(y_{[1..n-k]}) + w(k - \text{TZ}(y_{[n-k+1..n]})) + (1-w)\text{TZ}(y_{[n-k+1..n]}) \\ &= \text{LO}(y_{[1..n-k]}) + wk + (1-2w)\text{TZ}(y_{[n-k+1..n]}) \\ &< n-k + wk + (1-2w)k = n-wk = h_w^k(1^{n-k}0^k). \end{aligned}$$

If $w \in (1/2, 1]$, for any $y \neq 1^n$, if further $\text{LO}(y_{[1..n-k]}) \neq n-k$, noting that (26) holds for all $w \in [0, 1]$, we have

$$h_w^k(y) < n-k + (1-w)k < n-k + wk = h_w^k(1^n),$$

where the last inequality uses $1-w < w$ for $w > 1/2$. For the case when $\text{LO}(y_{[1..n-k]}) = n-k$, we have $\sum_{i=n-k+1}^n \prod_{j=1}^i y_j < k$, and thus

$$\begin{aligned} h_w^k(y) &= \text{LO}(y_{[1..n-k]}) + w \sum_{i=n-k+1}^n \prod_{j=1}^i y_j + (1-w)\text{TZ}(y_{[n-k+1..n]}) \\ &\leq \text{LO}(y_{[1..n-k]}) + w \sum_{i=n-k+1}^n \prod_{j=1}^i y_j + (1-w) \left(k - \sum_{i=n-k+1}^n \prod_{j=1}^i y_j \right) \\ &= n-k + (1-w)k + (2w-1) \sum_{i=n-k+1}^n \prod_{j=1}^i y_j \\ &< n-k + (1-w)k + (2w-1)k = n-k + wk = h_w^k(1^n). \end{aligned}$$

If $w = 1/2$, for any $y \notin S^*$, let $a = \text{LO}(y)$, then $\text{LO}(y_{[1..n-k]}) \neq n-k$ if $a < n-k$. Noting that (26) holds for all $w < 1$ including the case of $w = 1/2$, we know that $h_w^k(y) < h_w^k(1^{n-k}0^k)$. If $a \geq n-k$, let $\tilde{x} = 1^a 0^{n-a}$. Then we know that $\tilde{x} \in S^*$, $h_1^k(y) = h_1^k(\tilde{x})$, and $h_2^k(y) < h_2^k(\tilde{x})$. Thus

$$h_w^k(y) = wh_1^k(y) + (1-w)h_2^k(y) < wh_1^k(\tilde{x}) + (1-w)h_2^k(\tilde{x}) = h_w^k(\tilde{x}).$$

For any $y \in S^*$, since $\sum_{i=n-k+1}^n \prod_{j=1}^i y_j + \text{TZ}(y_{[n-k+1..n]}) = k$, we easily know that $h_w^k(y) = h_w^k(\tilde{x})$. $\square$

From Lemma 14, we easily obtain the following theorem to show the difficulty of the scalarization approach.

Theorem 5. Let M be defined as in Lemma 12 and $k > 2$. Let $S \subseteq \mathbb{R}$ be a set of w , and x_w be one global optimum of h_w^k . Let $F = \{h^k(x_w) \mid w \in S\}$. Then $M \not\subseteq F$.

7.3. Inconveniences of ϵ -constraint approach

This subsection discusses the performance of the ϵ -constraint approach to maximize LOTZ_k . Similar to Section 5, it reformulates LOTZ_k into the following problem.

$$\max h_1^k(x) = \text{LO}(x) \text{ s.t. } \text{LO}(x_{[1..n-k]}) + \text{TZ}(x_{[n-k+1..n]}) \geq \epsilon. \quad (27)$$

It is not difficult to see that no feasible solution exists for $\epsilon > n$. Among all feasible solutions, the optimal solution set is $\{1^n\}$ if $\epsilon \leq n-k$ and $\{1^{n-i}0^i\}$ if $\epsilon \in (n-k+i-1, n-k+i]$ for some $i \in [1..k]$. Hence, together with the Pareto front for LOTZ_k (Lemma 12), we know that for a set of constrained problems (27) with $\epsilon \in \{\epsilon_0, \dots, \epsilon_k\}$ where $\epsilon_i \in (n-k+i-1, n-k+i]$ for $i \in [1..k]$ and $\epsilon_0 \in (-\infty, n-k]$, the set of optimal solutions results in a full coverage for LOTZ_k .

7.3.1. Penalty ways with proper parameters

The exterior penalty way with coefficient $r > 0$ to solve (27) is to maximize

$$g(x) = h_1^k(x) + r \min\{0, h_2^k(x) - \epsilon\}. \quad (28)$$

As discussed before, the nonparameter penalty way is identical to the exterior one with $r = 1$.

Since this section serves as a brief discussion, unlike the complete calculation of the optimal solution set for the maximization of (6) in Section 5.2.1, here we only focus on the optimal solution set for some specific settings of ϵ and r . This is sufficient to show the possibility of using a non-MOEA way to properly solve the multi-objective LOTZ_k problem without distracting the readers. The following theorem shows the optimal solution set of the maximization of (28) with different settings of ϵ and r .

Theorem 6. Let $\varepsilon_i = n - k + i$ for $i \in [0..k]$. Consider the maximization of (28) with $\varepsilon = \varepsilon_i$ and a given value of r . If $r > 1$, the optimal solution set is $\{1^{n-i}0^i\}$. If $r = 1$, the optimal solution set is $\{1^{n-j}0^j \mid j \in [0..k]\}$.

Proof. If x is feasible, that is, $h_2^k(x) \geq \varepsilon_i = n - k + i$, then $\text{TZ}(x_{[n-k+1..n]}) \geq i$ as $\text{LO}(x_{[1..n-k]}) \leq n - k$. Hence, $g(x) = h_1^k(x) \leq n - \text{TZ}(x_{[n-k+1..n]}) \leq n - i$, where $g(x) = n - i$ holds if and only if $\text{TZ}(x_{[n-k+1..n]}) = i$ and $\text{LO}(x) = n - i$, that is, if and only if $x = 1^{n-i}0^i$. Hence among the feasible solutions, $x = 1^{n-i}0^i$ is the unique maximizer of (28).

Let now x be infeasible, that is, $h_2^k(x) < \varepsilon_i = n - k + i$. If $\text{LO}(x) < n - k$, then we have

$$g(x) = h_1^k(x) + r(h_2^k(x) - (n - k + i)) < n - k,$$

simply because $r > 0$. If $\text{LO}(x) \geq n - k$, then $\text{LO}(x_{[1..n-k]}) = n - k$ (that is, $x_{[1..n-k]} = 1^{n-k}$), thus $\text{LO}(x) = n - k + \text{LO}(x_{[n-k+1..n]})$ and $\text{TZ}(x_{[n-k+1..n]}) < i$ as x is infeasible. Together with $g(1^{n-i}0^i) = h_1^k(1^{n-i}0^i) = n - i$, for $r \geq 1$ we have

$$\begin{aligned} g(x) &= h_1^k(x) + r(h_2^k(x) - (n - k + i)) \\ &= n - k + \text{LO}(x_{[n-k+1..n]}) + g(1^{n-i}0^i) - (n - i) + r(\text{TZ}(x_{[n-k+1..n]}) - i) \\ &= g(1^{n-i}0^i) + \text{LO}(x_{[n-k+1..n]}) - k + i + r(\text{TZ}(x_{[n-k+1..n]}) - i) \\ &\leq g(1^{n-i}0^i) - \text{TZ}(x_{[n-k+1..n]}) + i + r(\text{TZ}(x_{[n-k+1..n]}) - i) \\ &= g(1^{n-i}0^i) + (r - 1)(\text{TZ}(x_{[n-k+1..n]}) - i) \leq g(1^{n-i}0^i), \end{aligned}$$

where the first inequality uses $\text{LO}(x_{[n-k+1..n]}) + \text{TZ}(x_{[n-k+1..n]}) \leq k$, and the second inequality uses $r \geq 1$ and $\text{TZ}(x_{[n-k+1..n]}) < i$ discussed above. This shows that $x = 1^{n-i}0^i$ is a maximizer of (28). The inequalities above are equalities only for $r = 1$ and $\text{LO}(x_{[n-k+1..n]}) + \text{TZ}(x_{[n-k+1..n]}) = k$, which completes our proof. $\square$

From Theorem 6, we obtain the following corollary to show that the possible difficulty of using the nonparameter penalty way (which is identical to the exterior penalty way with $r = 1$ as discussed before) to maximize (27).

Corollary 5. Let M be defined as in Lemma 12 and $k > 2$. For $i \in [0..k]$, let $\varepsilon_i = n - k + i$, and x_i be one maximizer of the nonparameter penalty function for the maximization of (27) with $\varepsilon = \varepsilon_i$. Let $F = \{h^k(x_i) \mid i \in [0..k]\}$. Then $M \subseteq F$ does not always hold.

Different from the difficulty of the nonparameter penalty function as in Corollary 5, from Theorem 6 and Lemma 12, we know that the exterior penalty way with $r > 1$ to solve (28) with $\varepsilon \in \{\varepsilon_i \mid i \in [0..k]\}$ will cover the full Pareto front of LOTZ_k . To obtain an overall performance for the further comparison with the MOEAs, now we consider the runtime of the randomized local search algorithm to maximize (28) with $\varepsilon = \varepsilon_i$ (for any given $i \in [0..k]$) and $r > 1$. The following two lemmas show the characteristics of the survivals from an infeasible solution and from a feasible solution respectively.

Lemma 15. Let S^* be defined as in Lemma 12. Let $i \in [0..k]$ and $\varepsilon_i = n - k + i$. Consider using the randomized local search algorithm to maximize g defined as in (28) with $\varepsilon = \varepsilon_i$ and $r > 1$. Let x be an infeasible solution, and y be the offspring generated by flipping the j th ($j \in [1..n]$) bit of x .

- (1) If $x \notin S^*$, y survives iff $j \in [\text{LO}(x) + 1..n - \text{TZ}(x_{[n-k+1..n]})]$. We have $g(y) > g(x)$ iff $j = \text{LO}(x) + 1$ or both $j = n - \text{TZ}(x_{[n-k+1..n]})$ and $\text{TZ}(x_{[n-k+1..n]}) < k$ hold;
- (2) If $x \in S^*$, y survives iff $g(y) > g(x)$, that is, iff $j = \text{LO}(x)$.

Proof. We first discuss the case that this infeasible x is not a Pareto optimum. In this case, we know that x has the form of $1^{\text{LO}(x)}0 * 10^{\text{TZ}(x_{[n-k+1..n]})}$ with $*$ $\in \{0, 1\}^{n-2-\text{LO}(x)-\text{TZ}(x_{[n-k+1..n]})}$ if $\text{TZ}(x_{[n-k+1..n]}) < k$, or the form of $1^{\text{LO}(x)}0 * 0^k$ with $*$ $\in \{0, 1\}^{n-k-1-\text{LO}(x)}$ if $\text{TZ}(x_{[n-k+1..n]}) = k$. In the second case, if in addition $x = 1^{n-k-1}0^{k+1}$, then we have $\text{LO}(x) + 1 = n - \text{TZ}(x_{[n-k+1..n]})$; in all other situations, we have $\text{LO}(x) + 1 < n - \text{TZ}(x_{[n-k+1..n]})$.

Since x is infeasible, we have $g(x) = h_1^k(x) + r(h_2^k(x) - \varepsilon)$. As x is not Pareto optimal, based on the form of x analyzed above, we know that a bit flip in the first $\text{LO}(x)$ positions gives $h_2^k(y) \leq h_2^k(x) \leq \varepsilon_i$ and $h_1^k(y) < h_1^k(x)$, and we know that a bit flip in the last $\text{TZ}(x_{[n-k+1..n]})$ positions gives $h_1^k(y) = h_1^k(x)$ and $h_2^k(y) < h_2^k(x) \leq \varepsilon_i$. Consequently, we have $g(y) < g(x)$ and y will not survive.

Now we consider the flip of the j th bit for $j \in [\text{LO}(x) + 1..n - \text{TZ}(x_{[n-k+1..n]})]$. Recall the already proved fact that $\text{LO}(x) + 1 < n - \text{TZ}(x_{[n-k+1..n]})$ (except that $\text{LO}(x) + 1 = n - \text{TZ}(x_{[n-k+1..n]})$ if $x = 1^{n-k-1}0^{k+1}$).

- If $j = \text{LO}(x) + 1$ (including $j = \text{LO}(x) + 1 = n - \text{TZ}(x_{[n-k+1..n]})$ if $x = 1^{n-k-1}0^{k+1}$), then $h_1^k(y) \geq h_1^k(x) + 1$ and $h_2^k(y) \geq h_2^k(x)$. Thus $g(y) \geq h_1^k(y) + r(h_2^k(y) - \varepsilon) > g(x)$, and y will survive.
- For $j \in [\text{LO}(x) + 2..n - \text{TZ}(x_{[n-k+1..n]}) - 1]$ (if this exists), we have $h^k(x) = h^k(y)$, and y will survive since $g(x) = g(y)$;
- Let $j = n - \text{TZ}(x_{[n-k+1..n]})$ (excluding the case when $x = 1^{n-k-1}0^{k+1}$). When $\text{TZ}(x_{[n-k+1..n]}) = k$, since $\text{LO}(x) + 1 < n - \text{TZ}(x_{[n-k+1..n]}) = n - k$, we know $h^k(x) = h^k(y)$, and y will survive. When $\text{TZ}(x_{[n-k+1..n]}) < k$, we have $h_1^k(y) = h_1^k(x)$ and $h_2^k(y) = h_2^k(x) + 1$, and then $g(y) \geq h_1^k(y) + r(h_2^k(y) - \varepsilon) > g(x)$. Thus y will survive.

Now it remains to discuss the case when x is a Pareto optimum. Then $\text{LO}(x) + \text{TZ}(x_{[n-k+1..n]}) = n$ and thus $\text{LO}(x) \geq n - k$. Since $\text{LO}(x) = n - k$ (that is, $x = 1^{n-k}0^k$) makes $h_2^k(x) = n \geq \varepsilon_i$ for any $i \in [0..k]$, we know that $x = 1^{n-k}0^k$ is feasible. Hence, in the following, we only consider $\text{LO}(x) > n - k$. When $j \in [1..n - k]$, we have $h^k(y) < h^k(x)$. Thus y is infeasible and $g(y) < g(x)$. When

$j \in [n - k..LO(x) - 1]$, we have $h_1^k(y) < h_1^k(x)$ and $h_2^k(y) = h_2^k(x)$. Thus y is infeasible and $g(y) < g(x)$. When $j = LO(x)$, we have $h_1^k(y) = h_1^k(x) - 1$ and $h_2^k(y) = h_2^k(x) + 1$. Then

$$g(y) \geq h_1^k(x) - 1 + r(h_2^k(x) + 1 - \varepsilon_i) = g(x) + r - 1 > g(x),$$

where the last inequality uses $r > 1$. Thus y will survive. When $j \in [LO(x) + 1..n]$ (if exists), we have $h_1^k(y) \leq h_1^k(x) + 1$ and $h_2^k(y) < h_2^k(x) < \varepsilon_i$, where the last inequality uses the fact that x is infeasible. Hence, y is infeasible and

$$g(y) = h_1^k(y) + r(h_2^k(y) - \varepsilon_i) \leq h_1^k(x) + 1 + r(h_2^k(x) - 1 - \varepsilon_i) = g(x) + 1 - r < g(x),$$

where the last inequality uses $r < 1$. $\square$

Lemma 16. Let $i \in [0..k]$ and $\varepsilon_i = n - k + i$. Consider using the randomized local search algorithm to maximize g defined as in (28) with $\varepsilon = \varepsilon_i$ and $r > 1$. Let x be a feasible solution and y be the offspring generated by flipping the j th ($j \in [1..n]$) bit of x .

- I. x is the unique optimum iff $LO(x) = n - i$, and no y can survive once the optimum x is reached;
- II. Conditional on that x is not the optimum, y survives iff ($j \in [LO(x) + 1..LO(x) + k - i]$, $LO(x) \leq n - k$), or ($j \in [LO(x) + 1..n - i]$, $LO(x) \in [n - k..n - i - 1]$);
- III. Conditional on that x is not the optimum, $g(y) > g(x)$ iff $j = LO(x) + 1$;
- IV. If y survives, y is feasible as well.

Proof. We first prove the fact I. For a feasible x with $LO(x) \geq n - k$, we know that $n - k + i \leq LO(x_{[1..n-k]}) + TZ(x_{[n-k+1..n]}) = n - k + TZ(x_{[n-k+1..n]})$. Hence $TZ(x_{[n-k+1..n]}) \geq i$, that is, $x_{[n-i+1..n]} = 0^i$. Thus $LO(x_{[n-k+1..n]}) \leq k - i$ and $LO(x) \leq n - k + k - i = n - i$. If further $LO(x) = n - i$, then with the above $x_{[n-i+1..n]} = 0^i$, we know that $x = 1^{n-i}0^i$, which is the unique optimum (for given $\varepsilon = \varepsilon_i$ and $r > 1$) from Theorem 6. As $x = 1^{n-i}0^i$ results in $LO(x) = n - i$, the first part of the fact I is proved. Since $x = 1^{n-i}0^i$ is the unique optimum as discussed and $y \neq x$ after the one-bit mutation, we know that no y will have $g(y) \geq g(x)$ and that no replacement will happen. Then the fact I is proved. Noting that $1^{n-k}0^k$ is the unique optimum and the only feasible solution for $i = k$, we only consider $i \in [0..k - 1]$ in the following.

We then prove the facts II to IV for a feasible x with $LO(x) \in [n - k..n - i - 1]$. Then $x_{[n-i+1..n]} = 0^i$ as discussed above. Since $j \in [1..LO(x)]$ gives $h_1^k(y) < h_1^k(x)$, we have $g(y) \leq h_1^k(y) < h_1^k(x) = g(x)$, and thus y will not survive. When $j = LO(x) + 1$, we have $h_1^k(y) \geq h_1^k(x) + 1 \geq n - k + 1$ and $y_{[n-i+1..n]} = x_{[n-i+1..n]} = 0^i$. Thus $h_2^k(y) \geq n - k + i$. Hence, y is feasible and $g(y) > g(x)$. When $j \in [LO(x) + 2..n - i]$ (if exists), $h_1^k(y) = h_1^k(x)$ and $h_2^k(y) \geq h_2^k(x) \geq n - k + i$. Thus y is feasible and $g(y) = g(x)$. When $j \in [n - i + 1..n]$, we know that $h_1^k(y) = h_1^k(x)$ and $h_2^k(y) \leq n - k + i - 1$. Hence, $g(y) = g(x) + r(h_2^k(y) - (n - k + i)) < g(x)$, and y will not survive. Thus, the facts III and IV and the last part of the fact II are proven for the case when the feasible x has $LO(x) \in [n - k..n - i - 1]$.

It remains to prove the facts II to IV when the feasible x has $LO(x) < n - k$. Then $LO(x) = LO(x_{[1..n-k]})$. Since x is feasible, we know that $LO(x) + TZ(x_{[n-k+1..n]}) \geq n - k + i$. Thus $LO(x) + k - i \geq n - TZ(x_{[n-k+1..n]})$ and then

$$x_{[LO(x)+k-i+1..n]} = 0^{n-LO(x)-k+i}. \quad (29)$$

When $j \in [1..LO(x)]$, we have $h_1^k(y) < h_1^k(x)$. Thus $g(y) \leq h_1^k(y) < h_1^k(x) = g(x)$ and y will not survive. When $j = LO(x) + 1$, which is also the case of $j = LO(x) + k - i$ for $i = k - 1$, then $h_1^k(y) \geq h_1^k(x) + 1$ and $h_2^k(y) \geq h_2^k(x) + 1 > n - k + i$, hence $g(y) > g(x)$. When $j \in [LO(x) + 2..LO(x) + k - i]$ for $i \in [0..k - 2]$, we know that $h_1^k(y) = h_1^k(x)$ and

$$h_2^k(y) = LO(y) + TZ(y_{[n-k+1..n]}) \geq LO(x) + n - LO(x) - k + i = n - k + i,$$

where the inequality uses (29). Hence, y is feasible and $g(y) = g(x)$. When $j \in [LO(x) + k - i + 1..n]$ for any $i \in [0..k - 1]$, with (29) we know that $h_1^k(y) = h_1^k(x)$ and

$$\begin{aligned} h_2^k(y) &= TZ(y_{[n-k+1..n]}) + LO(y_{[1..n-k]}) \\ &\leq (n - (LO(x_{[1..n-k]})) + k - i + 2) + LO(x_{[1..n-k]}) \\ &= n - k + i - 2 + 1 = n - k + i - 1 < \varepsilon_i. \end{aligned}$$

Thus y is infeasible and

$$g(y) = h_1^k(y) + r(h_2^k(y) - \varepsilon_i) = h_1^k(x) + r(h_2^k(y) - \varepsilon_i) < h_1^k(x) = g(x).$$

Thus, the facts III and IV and the first part of the fact II are proven for the case when the feasible x has $LO(x) < n - k$.

Noting that $LO(x) + k - i = n - k + k - i = n - i$ for $LO(x) = n - k$ w.r.t. the fact II, this lemma is proved. $\square$

Now we have the following runtime result.

Theorem 7. Let $i \in [0..k]$ and $\varepsilon_i = n - k + i$. Consider using the randomized local search algorithm to maximize g defined as in (28) with $\varepsilon = \varepsilon_i$ and $r > 1$. Then after $O(n^2)$ iterations in expectation, the optimal solution $1^{n-i}0^i$ is reached.

Proof. We pessimistically assume that x starts from an infeasible solution. From Lemma 15, we know that the flip in any position that makes the offspring survive will not decrease the value of h_2^k . We consider the flip in the j th position, where

$$\begin{cases} j = n - TZ(x_{[n-k+1..n]}), & \text{if } x \notin S^* \text{ and } TZ(x_{[n-k+1..n]}) < k, \\ j = LO(x) + 1, & \text{if } x \notin S^* \text{ and } TZ(x_{[n-k+1..n]}) = k, \\ j = LO(x), & \text{if } x \in S^*. \end{cases}$$

Also from Lemma 15, we know that such a flip will strictly increase the value of g , and also strictly increase the value of h_2^k . Since the probability to flip a specific bit is $1/n$, we know that an improvement of h_2^k will be witnessed in at most n expected number of iterations (function evaluations). Hence, to reach a feasible solution, that is, a solution with its h_2^k value at least $\varepsilon_i = n - k + i$, we need at most $n(n - k + i)$ expected number of function evaluations.

Once a feasible solution is reached, from Lemma 16 we know that all individuals in future iterations will be feasible (that is, $g = h_1^k$), and $h_1^k (= g)$ does not decrease. Also from Lemma 16, only the flip in the $(\text{LO}(x) + 1)$ th bit for the current individual x will result in the improvement of $h_1^k (= g)$. Hence, with the probability of $1/n$ to flip the $(\text{LO}(x) + 1)$ th bit, we know that in expected n iterations (function evaluations), an improvement of h_1^k will be witnessed. As the unique optimum is reached when $h_1^k = n - i$ (from Lemma 16 (1)), we know that the optimum is reached in at most $n(n - i)$ expected number of function evaluations.

Hence, this theorem is proved. $\square$

7.4. Efficiency of the MOEAs

For this brief discussion, we only consider the non-decomposition MOEAs ((G)SEMO, NSGA-II, and SMS-EMOA) as examples to see how the MOEAs maximize LOTZ_k . The following theorem gives the runtime result.

Theorem 8. Consider using (G)SEMO, SMS-EMOA with $\mu \geq k + 1$, or the NSGA-II with $N \geq 4(k + 1)$ to optimize the LOTZ_k problem. Then the expected number of function evaluations to cover the full Pareto front is $O(\max\{k, 1\}n^2)$ for (G)SEMO, $O(\mu n^2)$ for the SMS-EMOA, and $O(Nn^2)$ for the NSGA-II.

Proof sketch. The proof idea is similar to the one for the ONEMAXMIN_k problem. Here, we consider two phases of the optimization process. The first phase starts from the initial generation and ends when 1^n is reached for the first time. The second phase starts right after the end of the first phase and ends when the entire Pareto front is covered.

For the first phase, for some iteration t let $h_{1,\max}^k = \max_{z \in P_t} \{h_1^k(z)\}$ be the maximal h_1^k value in the population P_t . Note that Lemma 9 holds for all problems where the maximal number of mutually incomparable solutions is $k + 1$, thus also holds for LOTZ_k . Hence, $h_{1,\max}^k$ will not decrease, and will reach its maximum of n iff 1^n is reached. We pessimistically consider the following case when a strict improvement happens. In one iteration, one individual x in $A_t := \{z \mid h_1^k(z) = h_{1,\max}^k\}$ is selected as a parent (for the NSGA-II we further require $\text{cDis}(x) = +\infty$) and generates an offspring y with $h_1^k(y) > h_{1,\max}^k$ (which is easily reached when only the $(h_{1,\max}^k + 1)$ th bit of x is flipped). Then from Lemma 9, we know that in the next generation there is a z such that $z \geq y$, thus $h_1^k(z) \geq h_1^k(y) > h_{1,\max}^k$. Also from Lemma 9, we know that A_s for $s > t$ will not be empty before an improvement of $h_{1,\max}^k$ is witnessed. Noting that the probability of generating such a y when x is chosen for mutation is $\Omega(1/n)$, together with the probability of choosing x as a parent (discussed in Lemmas 8 and 10), we know that the expected number of function evaluations to see an improvement of $h_{1,\max}^k$ is $O(kn)$ for the GSEMO, $O(\mu n)$ for the SMS-EMOA, and $O(Nn)$ for the NSGA-II. Hence, to reach $h_{1,\max}^k = n$, that is, to reach 1^n as discussed before, the expected number of function evaluations that we need is $O(kn^2)$ for the GSEMO, $O(\mu n^2)$ for the SMS-EMOA, and $O(Nn^2)$ for the NSGA-II.

For the second phase, we pessimistically consider of sequentially generating $1^{n-1}0, \dots, 1^{n-k}0^k$. Similar to the above discussion, we only need to focus on selecting $1^{i+1}0^{n-i-1}$ as a parent to generate 1^i0^{n-i} with $i \in [n - k, n - 1]$. As there are at most k remaining missing Pareto front points when 1^n is reached for the first time, we know that the additional expected number of evaluations to cover the full Pareto front is $O(k^2n)$ for the GSEMO, $O(\mu nk)$ for the SMS-EMOA, and $O(Nnk)$ for the NSGA-II. $\square$

8. Conclusion

With the benchmark class ONEMAXMIN_k to depict different degrees of conflict between two objectives, we proved that the scalarization approach has difficulty in covering the full Pareto front, and that ε -constraint approach covers the full Pareto front in expected $O(\max\{k, 1\}n \ln n)$ function evaluations but the solving method and the parameter settings need to be carefully chosen.

In contrast to a good distribution of the set of parameter ε and the proper penalty coefficient r for the exterior penalty function way to successfully solve the corresponding ε -constraint problems, the MOEAs (including the basic (G)SEMO, and the modern NSGA-II, SMS-EMOA, and MOEA-D) can cover the full Pareto front in expected $O(\max\{k, 1\}n \ln n)$ function evaluations with less effort. The NSGA-II and SMS-EMOA (the MOEA/D) require, instead of a good distribution, the population size (the number of subproblems) at least a constant factor larger than the maximal size of incomparable set. This showed that the MOEAs easily achieve at least the same asymptotic runtime as the non-MOEA approaches, but without careful reformulation of the bi-objective problem. This also showed that even in the extreme case where two objectives are the same, the MOEAs have the same asymptotic runtime as solving the corresponding single-objective problem. We also saw the similar findings on a bi-objective LEADINGONES variant. It is the first work that theoretically compares the performance of the MOEAs and non-MOEA methods.

This paper demonstrated that the reasonable runtime of the MOEAs without decomposition is essentially guaranteed by two properties: (1) the ability to retain all reached Pareto front points and (2) an easy way to generate new Pareto front points from the already reached ones. For bi-objective optimization, both properties are easily satisfied: the dominance-based criterion in the survival selection, and the standard variation operator ensuring the reachability of all solutions. The reasonable runtime for the MOEA/D stems from a high-quality decomposition and a good runtime for each resulting single-objective subproblem. It is an interesting future research question about the comparison on the approximation guarantees when the reached Pareto front needs to be discarded for the MOEAs without decomposition, or when the decomposition employed by MOEA/D poorly aligns with the Pareto front. Besides,

the additional essential reason, in our discussions, for these MOEAs with the same runtime as the non-MOEAs with the deliberate designs on the parameters, lies in the fact that the maximal size of mutually incomparable solutions equals the degree of conflict. Whether this fact holds or not depends on the problems to solve. Hence, it is also an interesting research topic to discuss other types of problems to represent different degrees of conflict, and the performance comparison among the MOEAs and the non-MOEAs.

Data availability

No data was used for the research described in the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by National Natural Science Foundation of China (Grant No. 62306086) and Guangdong Basic and Applied Basic Research Foundation (Grant No. 2025A1515011936).

References

- [1] M. Laumanns, L. Thiele, E. Zitzler, E. Welzl, K. Deb, Running time analysis of multi-objective evolutionary algorithms on a simple discrete optimization problem, in: *parallel problem solving from nature*, Springer, 2002, pp. 44–53.
- [2] M. Laumanns, L. Thiele, E. Zitzler, Running time analysis of multiobjective evolutionary algorithms on pseudo-Boolean functions, *IEEE Trans. Evol. Comput.* 8 (2004) 170–182.
- [3] W. Zheng, Y. Liu, B. Doerr, A first mathematical runtime analysis of the non-dominated sorting genetic algorithm II (NSGA-II), in: *Conference on Artificial Intelligence*, AAAI 2022, AAAI Press, 2022, pp. 10408–10416.
- [4] W. Zheng, B. Doerr, Mathematical runtime analysis for the non-dominated sorting genetic algorithm II (NSGA-II), *Artif. Intell.* 325 (2023) 104016.
- [5] S. Wietheger, B. Doerr, A mathematical runtime analysis of the non-dominated sorting genetic algorithm III (NSGA-III), in: *International Joint Conference on Artificial Intelligence*, IJCAI 2023, Ijcai.org, 2023, pp. 5657–5665.
- [6] A. Opris, D.C. Dang, F. Neumann, D. Sudholt, Runtime analyses of NSGA-III on many-objective problems, in: *Genetic and Evolutionary Computation Conference*, GECCO 2024, ACM, 2024, pp. 1596–1604.
- [7] C. Bian, Y. Zhou, M. Li, C. Qian, Stochastic population update can provably be helpful in multi-objective evolutionary algorithms, *Artif. Intell.* 341 (2025) 104308.
- [8] Y.-L. Li, Y.-R. Zhou, Z.-H. Zhan, J. Zhang, A primary theoretical study on decomposition-based multiobjective evolutionary algorithms, *IEEE Trans. Evol. Comput.* 20 (2016) 563–576.
- [9] D. Antipov, T. Kötzing, A. Radhakrishnan, Greedy versus curious parent selection for multi-objective evolutionary algorithms, in: *Parallel Problem Solving from Nature*, PPSN 2024, Springer, 2024, pp. 86–101.
- [10] W. Zheng, B. Doerr, Runtime analysis for the NSGA-II: proving, quantifying, and explaining the inefficiency for many objectives, *IEEE Trans. Evol. Comput.* 28 (2024) 1442–1454.
- [11] B. Doerr, M.S. Krejca, N. Weeks, Proven runtime guarantees for how the MOEA/D: computes the pareto front from the subproblem solutions, in: *International Conference on Parallel Problem Solving from Nature*, PPSN 2024, Springer, 2024, pp. 197–212.
- [12] O. Giel, P.K. Lehre, On the effect of populations in evolutionary multi-objective optimisation, *Evol. Comput.* 18 (2010) 335–356.
- [13] C. Qian, Y. Yu, Z. Zhou, An analysis on recombination in multi-objective evolutionary optimization, *Artif. Intell.* 204 (2013) 99–119.
- [14] W. Zheng, B. Doerr, Theoretical analyses of multiobjective evolutionary algorithms on multimodal objectives, *Evol. Comput.* 31 (2023) 337–373.
- [15] W. Zheng, M. Li, R. Deng, B. Doerr, How to use the metropolis algorithm for multi-objective optimization?, in: *Conference on Artificial Intelligence*, AAAI 2024, AAAI Press, 2024, pp. 20883–20891.
- [16] O. Giel, Expected runtimes of a simple multi-objective evolutionary algorithm, in: *Congress on Evolutionary Computation*, CEC 2003, IEEE, 2003, pp. 1918–1925.
- [17] D. Brockhoff, T. Friedrich, F. Neumann, Analyzing hypervolume indicator based algorithms, in: *Parallel Problem Solving from Nature PPSN 2008*, Springer, 2008, pp. 651–660.
- [18] B. Doerr, B. Kodric, M. Voigt, Lower bounds for the runtime of a global multi-objective evolutionary algorithm, in: *Congress on Evolutionary Computation*, CEC 2013, IEEE, 2013, pp. 432–439.
- [19] B. Doerr, W. Gao, F. Neumann, Runtime analysis of evolutionary diversity maximization for OneMinMax, in: *Genetic and Evolutionary Computation Conference*, ACM, 2016, pp. 557–564.
- [20] C. Bian, C. Qian, K. Tang, A general approach to running time analysis of multi-objective evolutionary algorithms, in: *International Joint Conference on Artificial Intelligence*, IJCAI 2018, Ijcai.org, 2018, pp. 1405–1411.
- [21] Z. Huang, Y. Zhou, Z. Chen, X. He, Running time analysis of MOEA/D with crossover on discrete optimization problem, in: *Conference on Artificial Intelligence*, AAAI 2019, AAAI Press, 2019, pp. 2296–2303.
- [22] Z. Huang, Y. Zhou, Runtime analysis of somatic contiguous hypermutation operators in MOEA/D framework, in: *Conference on Artificial Intelligence*, AAAI 2020, AAAI Press, 2020, pp. 2359–2366.
- [23] E.C. Osuna, W. Gao, F. Neumann, D. Sudholt, Design and analysis of diversity-based parent selection schemes for speeding up evolutionary multi-objective optimisation, *Theor. Comput. Sci.* 832 (2020) 123–142.
- [24] C. Bian, C. Qian, Better running time of the non-dominated sorting genetic algorithm II (NSGA-II) by using stochastic tournament selection, in: *Parallel Problem Solving from Nature*, PPSN 2022, Springer, 2022 pp. 428–441.
- [25] W. Zheng, B. Doerr, Approximation guarantees for the non-dominated sorting genetic algorithm II (NSGA-II), *IEEE Trans. Evol. Comput.* 29 (2025) 891–905.
- [26] D. Dang, A. Opris, D. Sudholt, Crossover can guarantee exponential speed-ups in evolutionary multi-objective optimisation, *Artif. Intell.* 330 (2024), 104098.
- [27] D. Dang, A. Opris, D. Sudholt, Illustrating the efficiency of popular evolutionary multi-objective algorithms using runtime analysis, in: *Genetic and Evolutionary Computation Conference*, GECCO 2024, ACM, 2024.
- [28] W. Zheng, Theoretical advantage of multiobjective evolutionary algorithms for problems with different degrees of conflict, 2024. <https://arxiv.org/abs/2408.04207>.
- [29] M. Caramia, P. Dell’Olmo, Multi-objective optimization, in: *Multi-objective Management in Freight Logistics: Increasing Capacity, Service Level, Sustainability, and Safety with Optimization Algorithms*, Springer, 2020, pp. 21–51.
- [30] I. Das, J.E. Dennis, A closer look at drawbacks of minimizing weighted sums of objectives for pareto set generation in multicriteria optimization problems, *Struct. Optim.* 14 (1997) 63–69.

- [31] K. Deb, Multi-Objective Optimization Using Evolutionary Algorithms, 16, John Wiley & Sons, 2001.
- [32] C.A.C. Coello, Evolutionary Algorithms for Solving Multi-objective Problems, Springer, 2007.
- [33] Y. Zhou, J. He, A runtime analysis of evolutionary algorithms for constrained optimization problems, *IEEE Trans. Evol. Comput.* 11 (2007) 608–619.
- [34] K. Deb, An efficient constraint handling method for genetic algorithms, *Comput. Methods Appl. Mech. Eng.* 186 (2000) 311–338.
- [35] Q. Zhang, H. Li, MOEA/D: a multiobjective evolutionary algorithm based on decomposition, *IEEE Trans. Evol. Comput.* 11 (2007) 712–731.
- [36] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Trans. Evol. Comput.* 6 (2002) 182–197.
- [37] B. Doerr, Z. Qu, A first runtime analysis of the NSGA-II on a multimodal problem, *IEEE Trans. Evol. Comput.* 27 (2023) 1288–1297.
- [38] B. Doerr, Z. Qu, From understanding the population dynamics of the NSGA-II to the first proven lower bounds, in: *Conference on Artificial Intelligence, AAAI 2023*, AAAI Press, 2023, pp. 12408–12416.
- [39] S. Cerf, B. Doerr, B. Hebras, J. Kahane, S. Wietheger, The first proven performance guarantees for the non-Dominated sorting genetic algorithm II (NSGA-II) on a combinatorial optimization problem, in: *International Joint Conference on Artificial Intelligence, IJCAI 2023*, Ijcai.org, 2023, pp. 5522–5530.
- [40] D.-C. Dang, A. Opris, B. Salehi, D. Sudholt, Analysing the robustness of NSGA-II under noise, in: *Genetic and Evolutionary Computation Conference, GECCO 2023*, ACM, 2023 pp. 642–651.
- [41] D. Dang, A. Opris, D. Sudholt, Level-based theorems for runtime analysis of multi-objective evolutionary algorithms, in: *International Conference on Parallel Problem Solving from Nature, PPSN 2024*, Springer, 2024, pp. 246–263.
- [42] R. Deng, W. Zheng, M. Li, J. Liu, B. Doerr, Runtime analysis for state-of-the-art multi-objective evolutionary algorithms on the subset selection problem, in: *International Conference on Parallel Problem Solving from Nature, PPSN 2024*, Springer, 2024, pp. 264–279.
- [43] C. Bian, S. Ren, M. Li, C. Qian, An archive can bring provable speed-ups in multi-objective evolutionary algorithms, in: *International Joint Conference on Artificial Intelligence, IJCAI 2024*, Ijcai.org, 2024, pp. 6905–6913.
- [44] T. Lu, C. Bian, C. Qian, Towards running time analysis of interactive multi-objective evolutionary algorithms, in: *Conference on Artificial Intelligence, AAAI 2024*, AAAI Press, 2024, pp. 20777–20785.
- [45] N. Beume, B. Naujoks, M. Emmerich, SMS-EMOA: Multiobjective selection based on dominated hypervolume, *Eur. J. Oper. Res.* 181 (2007) 1653–1669.
- [46] W. Zheng, B. Doerr, Runtime analysis of the SMS-EMOA for many-objective optimization, in: *Conference on Artificial Intelligence, AAAI 2024*, AAAI Press, 2024, pp. 20874–20882.